ARRP
Persistent Automation

Technical Specification and Traceability Map

NON-AUTHORITATIVE REFERENCE PRODUCT

Describes the reviewed system; creates no authority, schedule, permission, or project decision.

Version 1.3
July 25, 2026

American Restoration and Resilience Project

Reviewed implementation baseline: `3ba710868a9aaa0a732c6e0df8a387f54cf2f504`

This PDF is generated from the repository's non-authoritative Markdown reference. The Framework, Agent Operating Rules, routed modules, registered runbooks, and reviewed runtime configuration remain the controlling sources.

Contents

This table is generated from the document's section structure. Open the source reference on GitHub.

Status and authority 5

1. Purpose, scope, and design goals 5

2. Terms and stable identities 6

3. Architecture at a glance 6

4. Authority and responsibility 7

4.1 Authority hierarchy . . . 7
4.2 Human-reserved decisions . . . 7
4.3 Delegated LLM authority . . . 8
4.4 Deterministic authority . . . 8
4.5 Coordinator and Console authority . . . 8

5. Registered component catalog 8

5.1 Persistent roles . . . 8
5.2 Worker contracts . . . 9
Run Coordinator Bot . . . 9
Case Monitor Bot . . . 9
Presidential Directives Bot . . . 9
Source Checker Bot . . . 9
Project Console Progress Bot . . . 9
Project Integrity Bot . . . 10
Elim . . . 10

6. Trigger and cadence model 10

7. End-to-end run chain 11

7.1 Cloud planning . . . 11
7.2 Deterministic stages . . . 11
7.3 Queue and context assembly . . . 12
7.4 Host dispatch . . . 12
7.5 Host activation and Elim task lifecycle . . . 13
7.6 Trigger-to-process invocation map . . . 14
7.7 Chronological transaction ledger . . . 15
7.8 Per-phase file and state ledger . . . 18
7.9 Lock, lease, and false-lock ledger . . . 20

7.10 Audit-handoff ownership and exact transitions . . . 21
7.11 Log and accounting matrix . . . 23

8. Concurrency, workspace isolation, and repository safety **23**

9. Stage status and failure semantics **25**

10. Work-queue contract **26**

11. Context gateway **27**

12. Model routing and usage protection **29**

13. Elim work order and result contract **29**

14. Public-intake pipeline **30**

15. Review Epochs **31**

16. GitHub, branch, and write boundaries **31**

17. Data, provenance, and retention **32**

18. Console and local administration **33**

19. Security and privacy controls **34**

20. Validation and acceptance **34**

21. Change control, deployment, and rollback **35**

22. Operational playbooks **36**

22.1 Healthy no-op chain . . . 36
22.2 Deterministic watcher finds a change . . . 36
22.3 Source URL fails . . . 36
22.4 Public submission arrives . . . 36
22.5 Usage approaches the reserve . . . 36
22.6 Interactive checkout is dirty . . . 36
22.7 Elim terminates unexpectedly . . . 36
22.8 Human decision is required . . . 37

23. Implementation status and known limitations **38**

Appendix A. File and component crosswalk **39**

Appendix B. Core invariants **39**

Appendix C. Glossary of stop conditions **40**

Appendix D. Project Integrity Bot check inventory 41

Status and authority

This document is a descriptive, integrative technical specification of ARRP's persistent automation. It does not create substantive, operational, or security authority; enable an agent or bot; alter a schedule, model, permission, or GitHub Project field; or supersede the canonical records it cites.

The Framework and its routed modules govern project substance and change control. Agent Operating Rules govern common execution. Each registered runbook governs its persistent role. GitHub Workflow governs GitHub Issues and Project fields. The Public-Intake Review Process governs contributor-content handling. Project Interface governs Console controls. `context-routes.json` governs machine context routing.

Reviewed `.github/*.json` manifests, workflows, schemas, and scripts are deployed implementations that must conform to those authorities. Logs, reports, workflow artifacts, run-chain data, context packets, and Console screens are evidence or derived views, not grants of authority. If this specification differs from a canonical authority or deployed implementation, the system must fail closed, report the drift, and correct the owning source through the required review. It must not choose whichever version permits an action.

The PDF edition is generated from this Markdown source. Both are non-authoritative snapshots and may become stale after a later architecture change.

1. Purpose, scope, and design goals

ARRP uses persistent automation to perform repeatable observation, validation, routing, and carefully bounded development without replacing human control over the project's defining judgments. The system exists to preserve interactive LLM capacity for comprehensive issue development while ensuring routine checks occur in a consistent order and failures become visible.

The architecture has six principal goals:

1. **Serialize automation.** Persistent workers run through one due-aware chain instead of independent overlapping schedules.
2. **Prefer deterministic work.** Scripts perform objective discovery, comparison, validation, classification, and rendering before an LLM is considered.
3. **Launch Elim conditionally.** The LLM agent receives one exact selected unit, a source-bound context packet, and an official usage attestation only when eligible work exists.
4. **Preserve human authority.** A queue, model, script, score, or successful check cannot create authority that the governing record withholds.
5. **Make every boundary auditable.** Chain IDs, work-unit IDs, source revisions, hashes, logs, pull requests, and continuation state connect observation to action.
6. **Fail safely.** Missing credentials, stale inputs, contradictory configuration, invalid context, uncertain authority, failed validation, and unsafe repository state stop or degrade the appropriate portion of the chain without absorbing user work.

Automated context, one-unit selection, and usage minimization apply to persistent automation. They do not impose a brevity or single-treatment limit on direct, interactive work between the user and Codex. Human-directed development remains task-shaped and comprehensive: it loads the additive union of every implicated module and may combine multiple methods, audits, issue treatments, or parallel reviews when the work calls for them.

2. Terms and stable identities

Term	Meaning
Human	The project owner or another expressly authorized human reviewer.
Interactive agent	An LLM session working directly with a human. It is not a persistent scheduled role merely because it uses the same model family.
Persistent LLM agent	A registered, repeatedly invoked LLM role with one authoritative runbook. Elim is the current persistent LLM agent.
Deterministic bot	A registered script or program whose operation does not require LLM judgment. Its stable name ends in `-bot`.
Chain stage	A serialized unit orchestrated by the Run Coordinator. Public intake is a stage, not a bot.
Chain ID	The stable identifier joining one coordinator plan, stage results, queue, context packet, usage record, Elim invocation, and closeout.
Work-unit ID	The stable identity of one selected actionable unit.
Canonical record	A project record that owns authoritative content or state.
Projection	Generated data derived from canonical records, such as a Console feed, queue, report, or context packet.
Material unit	A unit that changes a project or external record, or records and routes a material finding.
Proposed event	A bot-observed change that has not been accepted through human review or the governing merge boundary.
Accepted event	A reviewed change whose pull request or other authorized integration has been accepted.
Review Epoch	A point-in-time comprehensive consistency review of the registered governing boundary and unresolved findings.

ARRP reserves the word **bot** for deterministic programs. An LLM-directed worker does not receive a `-bot` suffix. Elim is therefore **Elim**, not “Elim Bot.”

3. Architecture at a glance

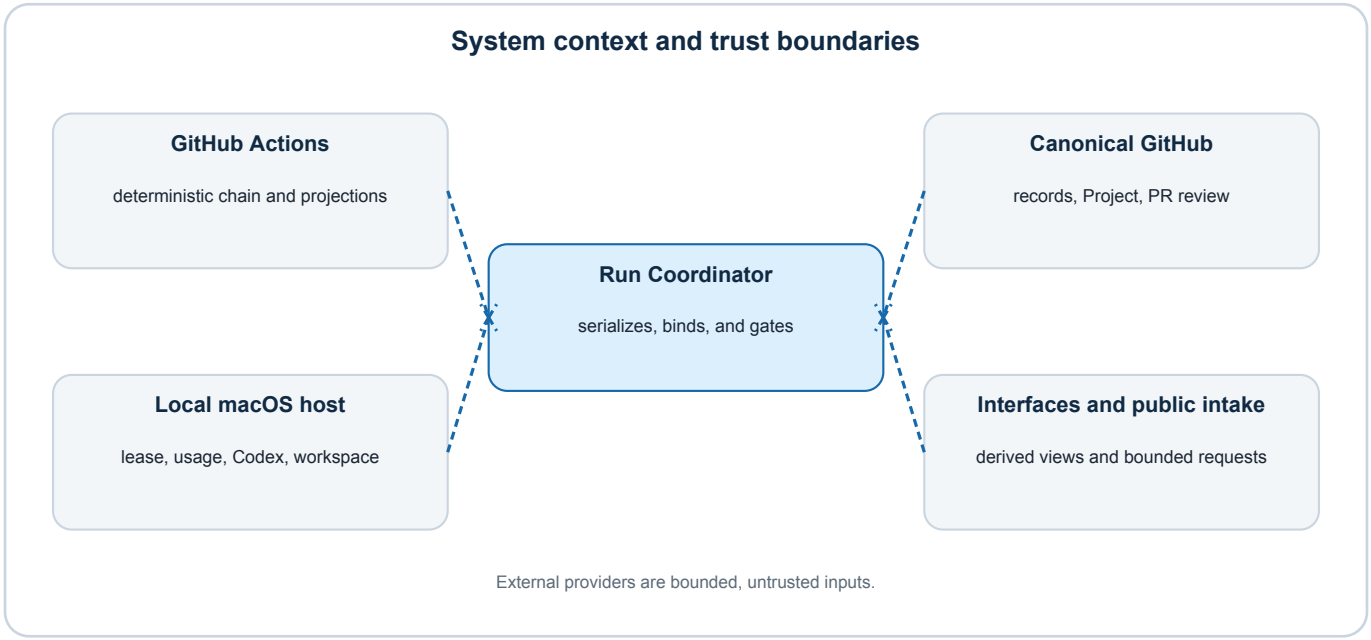

Figure: System context and trust boundaries. Arrows show data or control flow, not grants of authority.

The system spans four execution and trust domains:

- **GitHub Actions** performs deterministic planning, due-stage execution, queue preparation, context preparation, and generated-data publication.
- **The canonical GitHub repository and Project** own substantive records, workflow fields, reviewed configuration, branches, and pull requests.
- **The local macOS host** owns the exclusive dispatcher lease, the official Codex-usage reading, the localhost control service, the dedicated Elim workspace, and the Codex process.
- **Project interfaces and public services** display derived state or submit bounded requests. They never become substantive authority.

External provider APIs—including court data, Federal Register data, cataloged source URLs, and the first-party Codex rate-limit service—are treated as untrusted or availability-limited inputs. Provider responses may support observation but never expand project authority.

4. Authority and responsibility

4.1 Authority hierarchy

The routing order is:

1. root `AGENTS.md` bootstrap;
2. compact Framework routing kernel;
3. Agent Operating Rules;
4. operation-specific Framework and agent-rule modules;
5. the selected persistent-role runbook;
6. GitHub Workflow, Public Intake, Project Interface, or other owning specialized authority when implicated;
7. reviewed runtime configuration, workflow, schema, and code that implements those rules;
8. generated queues, packets, reports, logs, and Console views as evidence.

Lower layers may narrow behavior or implement a rule. They may not contradict or enlarge higher authority. A machine-selected route is not permission to act.

4.2 Human-reserved decisions

Only a human may permanently remove, retire, reject, admit, promote, merge, split, defer, or otherwise finally dispose of a candidate or issue. Human decision is also required to:

- answer the reversed-control question, “Would we want this institutional design if our least-favored political opponent controlled it?”;
- create or materially change the institutional failure, essential boundaries, remedy, vehicle, scope, fiscal architecture, or foundational premise of a proposal;
- change the Framework, methodology, audit rubric, score components, weights, penalties, thresholds, or score bands;
- adopt consequential external review that requires a reserved change;
- authorize final circulation or publication;
- moderate, delete, conceal, or contact a public contributor;
- admit a preliminary record as a formal candidate or a candidate as an issue; or
- merge a bot or agent pull request or change persistent automation configuration.

Agents must not ignore these matters. They may inspect them, identify the exact decision required, develop neutral alternatives, assess evidence, and make a reasoned recommendation for human review. A record-specific human decision permits only the identified implementation. It is not blanket or standing authority.

4.3 Delegated LLM authority

Inside an approved proposal foundation, Elim may conduct research, source development, causal analysis, neutral alternative-control analysis, drafting, audit work, and ordinary lifecycle maintenance. It may determine whether an existing canonical foundation satisfies the four established foundation gates and, when the record supports that conclusion, advance an issue to `In development`.

For a formal candidate, Elim may conduct the defined source-development and investigation task and recommend a disposition. It may not supply the human reversed-control answer, admit the candidate, create its formal proposal foundation, or implement a permanent disposition.

4.4 Deterministic authority

Bots may observe, compare, verify, classify, render, and route only what their runbooks define. A deterministic check may confirm that a required field exists and uses an allowed value. It may not decide that an institutional diagnosis, remedy, source identity substitution, or policy judgment is substantively adequate unless an expressly deterministic rule makes that conclusion objective.

4.5 Coordinator and Console authority

The Run Coordinator schedules and routes. It does not make legal, factual, lifecycle, scoring, publication, or disposition judgments.

The Console is a derived management interface. An authenticated localhost control may submit a request for Run Coordinator evaluation. It may not directly invoke or select an agent, bypass due, authority, context, repository, or usage gates, mutate canonical records, guarantee that a run will occur, or treat a request as a substantive decision.

5. Registered component catalog

5.1 Persistent roles

Stable ID	Type	Current posture	Primary function
`run-coordinator-bot`	Deterministic bot	Enabled	Serialize the chain, build verified inputs, apply host gates, and conditionally dispatch Elim.
`case-monitor-bot`	Deterministic bot	Enabled	Compare configured litigation sources and provider records for machine-observable developments.
`presidential-directives-bot`	Deterministic bot	Enabled	Compare the directive registry with official Federal Register metadata.
`source-checker-bot`	Deterministic bot	Report-only pilot	Check cataloged URLs and classify reachability and identity posture.
`project-console-progress-bot`	Deterministic bot	Enabled	Reconcile Project and registry records and publish progress projections.
`project-integrity-bot`	Deterministic bot	Enabled	Perform the final deterministic consistency and configuration check.
`elim`	Persistent LLM agent	Enabled, conditional	Author and validate the selected bounded development, audit, candidate, intake, or comprehensive-review unit; trusted-host code owns repository Git closeout.

Public intake is a permanent chain stage but is not a named bot. The local host dispatcher and localhost control service are persistent runtime components but are not independent substantive workers.

5.2 Worker contracts

Run Coordinator Bot

- **Authority:** orchestration only.
- **Cloud runtime:** `.github/workflows/run-coordinator-bot.yml`.
- **Host runtime:** `scripts/run_chain_dispatcher.py`.
- **Configuration:** `.github/run-coordinator-bot.json`.
- **Triggers:** daily schedule, manual dispatch, approved repository events, and deterministic-only `main` pushes.
- **Writes:** run-chain and bounded input projections on `project-console-data`; local host state under `.tmp/run-coordinator`; no substantive issue judgment.
- **Failure posture:** stage-specific blocking or degraded outcomes; host failures preserve a local failure event and notification and, whenever the dispatcher can safely own shared state, also route to local health history and Action Items.

Case Monitor Bot

- **Authority:** objective source observation only.
- **Due predicate:** every 24 hours in the chain, or manual chain request.
- **Inputs:** configured source-catalog rows, accepted baselines, Just Security tracker data, and bounded CourtListener queries.
- **Writes:** only allowed machine-observed fields and marker-bounded generated sections on its dedicated bot branch.
- **Acceptance:** human review and merge of its pull request.
- **Limit:** coverage is not exhaustive discovery and observed activity is not a legal-significance conclusion.

Presidential Directives Bot

- **Authority:** official-metadata comparison only.
- **Due predicate:** every 24 hours in the chain, or manual chain request.
- **Inputs:** the directive registry, accepted fingerprints, and the Federal Register API.
- **Writes:** authorized deterministic registry metadata on its persistent watcher branch.
- **Acceptance:** human review and merge.
- **Limit:** it does not decide relevance, institutional significance, issue routing, or proposal effects.

Source Checker Bot

- **Authority:** report-only URL and identity classification.
- **Due predicate:** every 168 hours, or manual chain request.
- **Inputs:** both source catalogs and bounded prior results.
- **Writes:** `project-console-data/source-checker.json`, retained artifact, and a replaceable Markdown report proposal when the current report changes.
- **Classifications:** verified; identity-preserving redirect; access restricted; transient failure; broken; identity mismatch; review required.
- **Limit:** it never substitutes a source, edits a supported proposition, or treats access restriction as breakage.

Project Console Progress Bot

- **Authority:** derived progress calculation only.
- **Due predicate:** every 24 hours; the coordinator also forces a deterministic refresh after relevant `main` pushes.
- **Inputs:** GitHub Project, issue registry, canonical development-level vocabulary, score and review-readiness rules.
- **Writes:** generated progress and bounded history on `project-console-data`.
- **Failure posture:** missing Project credentials, invalid Project structure, unknown development levels, incomplete joins, or publication failure fail closed.

Project Integrity Bot

- **Authority:** observation and routing only.
- **Due predicate:** every chain.
- **Inputs:** repository, registries, Project, Issues, publication state, worker registry, runbooks, manifests, workflows, and machine schemas.
- **Writes:** `project-console-data/integrity.json` and a replaceable current-report pull request.
- **Position in chain:** final deterministic stage, so Elim sees the most current deterministic exception set.
- **Limit:** a zero-finding report is current evidence, not permanent proof of correctness.

Elim

- **Authority:** delegated LLM development within the selected unit and governing boundaries.
- **Trigger:** only a finalized eligible chain and passing host gates.
- **Workspace:** the fixed dispatcher-managed full checkout at `.tmp/run-coordinator/elim-checkout`; Elim receives working-file access while its Git metadata and closeout remain trusted-host responsibilities, and it must not consume or rewrite interactive user changes.
- **Inputs:** the chain manifest, exact queue selection, bound context packet, preserved deterministic inputs, host usage attestation, and canonical records.
- **Writes:** only working-tree and authorized semantic GitHub records required by the selected unit. Elim declares the exact file set; the host validates, commits, synchronizes without force, and reads back the repository boundary.
- **Logs:** one Elim Run Log entry for every invocation; shared Agent Audit provenance for each material unit; detailed audit findings in the issue audit sidecar.

6. Trigger and cadence model

Trigger	Deterministic stages	May authorize Elim?	Notes
Daily schedule	All due stages	Yes	One coordinator kickoff at 04:17 UTC; local time varies with daylight saving time.
Manual coordinator request	Due or expressly requested chain work	Yes	Still subject to every gate.
Public-submission event	Reconciliation and due stages	Yes, if an eligible intake or other unit exists	Event contains no private submission body.
Forced comprehensive request	Due stages plus full-context Review Epoch unit	Yes	Comprehensive unit controls selection and context.
Governing-boundary change	Due stages plus off-cycle Review Epoch	Yes	Exact registered hash/set drift makes the epoch due.
Push to `main`	Deterministic refresh and integrity	No	Push is always LLM-forbidden.
Direct raw worker dispatch	That worker only	No direct Elim authority	Diagnostic/manual worker execution does not by itself become a full chain.

The chain uses UTC for scheduling and ISO timestamps for records. Interfaces may display local time, but daylight-saving changes do not alter the UTC schedule or due predicates.

Closely related eligible triggers are serialized by the GitHub concurrency group and the host lease. A healthy `not_due` or no-op result is expected and does not create a material log entry.

7. End-to-end run chain

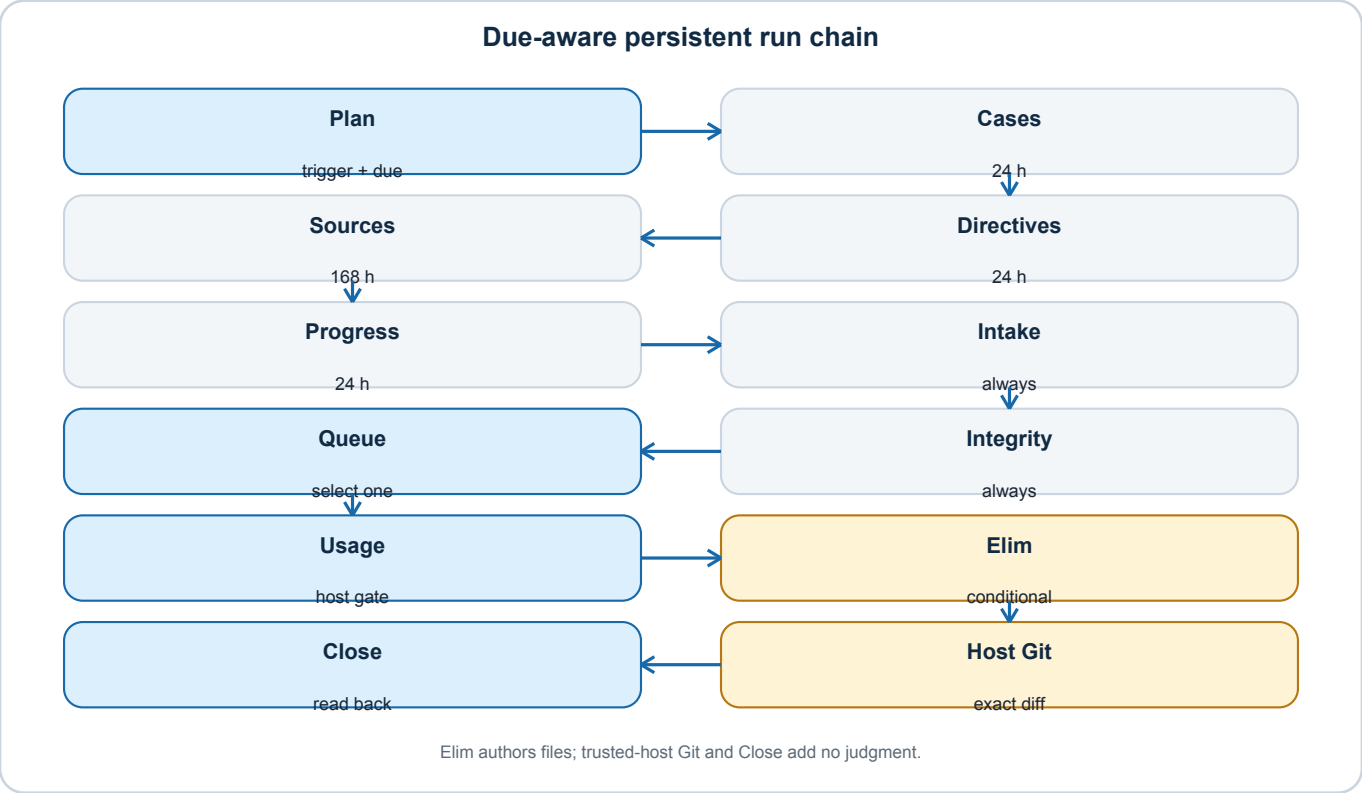

Figure: Due-aware persistent run chain. Arrows show data or control flow, not grants of authority.

7.1 Cloud planning

The coordinator:

1. checks out the reviewed `main` revision;
2. retrieves the previous chain boundary from `project-console-data`;
3. reads trigger flags and the latest Review Epoch;
4. validates the registered governing-file hashes;
5. computes due predicates;
6. creates a chain plan with one Chain ID; and
7. retains the planned manifest as a GitHub Actions artifact.

7.2 Deterministic stages

The current serialized order is:

1. Case Monitor Bot;

2. Presidential Directives Bot;
3. Source Checker Bot;
4. Project Console Progress Bot;
5. public-intake collection and reconciliation; and
6. Project Integrity Bot.

The Chain Manifest binds every stage to the chain-level baseline revision. For each stage it records the due disposition, bounded attempt count and retry disposition, completion time, output location and hash where applicable, work count, and public outcome. The Case Monitor, Presidential Directives, and Source Checker primary and retry invocations carry distinct caller-supplied attempt keys; the close job retrieves only the selected successful watcher artifact and verifies its report hash before materialization instead of choosing whichever file happens to be newest. A missing, malformed, oversized, schema-invalid, undated, stale, materially future-dated, or explicitly unavailable persistent watcher input forces its owning watcher due even when the ordinary success interval remains current. The coordinator validates each watcher against its typed report shape and the same configured interval that governs its due calculation. A proposed source-domain event is additionally bound by Event ID, content hash, Chain ID, exact Actions run attempt, and proposal revision. A missing or mismatched successful watcher artifact fails the chain; it is never replaced by an older current feed or synthetic empty input. Progress and Integrity use their own bounded retry outputs; public intake has one attempt. A deterministic stage may be `completed`, `not_due`, `degraded`, or `failed`. `Blocked` applies to the overall chain, host, work unit, or Elim continuation, not as a synthetic deterministic-stage state.

The case monitor, directive monitor, progress, and integrity stages are blocking when due and unavailable after retry. Source checking and public intake may degrade without preventing unrelated work because an access-limited source or intake service need not make all other project records unreliable. A degraded input remains visible and may restrict work that depends on it.

7.3 Queue and context assembly

After deterministic stages, the close job:

1. compiles stage outcomes;
2. preserves exact hashed copies of integrity, progress, intake, Review Epoch, chain, and registered watcher/source inputs;
3. builds and validates the versioned work queue;
4. selects one provisional eligible unit;
5. chooses the required context profile;
6. constructs a source-bound context packet; and
7. binds the queue selection and packet to the same revision and identity.

Local user suppressions or reprioritization are deliberately not applied by the cloud close job. The host rebuilds the queue and context from the verified inputs immediately before launch, applies the latest locked local controls there, and records the resulting exact selection.

A due comprehensive-review unit overrides ordinary eligible work and forces the comprehensive context profile, except that an expressly eligible safety-class-0 repair of the failed deterministic chain runs first so the comprehensive review does not begin from a known broken automation boundary.

7.4 Host dispatch

The local dispatcher:

1. acquires the operating-system lease;

2. fetches `origin/main`, requires the canonical checkout to be on `main` with local `HEAD` exactly equal that revision, automatically commits and fast-forward pushes ordinary uncommitted paths before deferring to the next poll, then verifies that the local dispatcher runtime byte-matches the synchronized revision and creates or refreshes the isolated Elim workspace;
3. downloads or retrieves the matching manifest and exact deterministic inputs;
4. mirrors the manifest, inputs, queue, context, usage, and result paths into the isolated checkout;
5. independently rehashes the inputs and rebuilds the queue and packet there, applying current user overrides and durable recovery state before final selection;
6. takes the official first-party usage reading and creates a unique invocation baseline inside the isolated boundary;
7. refinalizes the launch decision without discarding completed or degraded stage outcomes;
8. enforces trigger permission;
9. launches or resumes Elim only if an eligible unit remains;
10. refreshes usage and the diagnostic heartbeat while Elim runs;
11. validates the model-authored structured result against the exact selected unit, exact changed-path set, and required durable records;
12. refetches the approved remote and requires the checkout and `origin/main` to remain at the pinned baseline;
13. creates a bounded branch, stages only the declared paths, commits under the coordinator identity, and performs non-forced synchronization;
14. reads back the resulting `main` boundary or open human-review pull request and records the actual commit in the host result and local projection;
15. records host success or failure, alerts, bounded history, and continuation; and
16. releases the lease.

GitHub Actions never launches Codex. The approved local dispatcher is the only LLM launch boundary.

7.5 Host activation and Elim task lifecycle

The repository contains reviewed launchd templates. The installed dispatcher job polls every 600 seconds with `--launch-codex`; the control service starts at login and remains available on the localhost port. Installed plists live outside Git and are host state, not project authority. A local health check must therefore distinguish the reviewed template, the installed configuration, whether the job is loaded, whether a process is currently live, its last exit status, and the latest successful Chain ID.

The dispatcher preserves the first validated Elim Codex task identifier and resumes that exact task on later invocations. It never uses a generic “most recent task” shortcut. The current verified manifest and packet remain the sole active work authority; earlier task messages are historical context and may not supply current freshness, authority, or an additional work unit. This limits task-list clutter while keeping repository-owned logs as the operational record. Task archiving is reversible interface housekeeping, not a success or authority condition; the dispatcher must not discard a reusable task identifier merely because a UI archive operation is unavailable.

7.6 Trigger-to-process invocation map

The following table identifies the actual entry points. A trigger does not bypass due, authority, repository, context, or usage gates.

Trigger or caller	Technical invocation	Immediate state touched	Elim authority
Daily cloud schedule	GitHub invokes `.github/workflows/run-coordinator-bot.yml` at `17 4 * * *` (04:17 UTC).	GitHub Actions run, `arrp-run-chain` concurrency group, runner-temporary plan files, and the `run-chain-plan` artifact.	Yes, after the host consumes the completed eligible chain.
Public-submission or administrative repository event	GitHub receives `repository_dispatch` type `arrp-public-submission` or `arrp-run-chain` and invokes the same workflow.	The workflow event, privacy-minimized intake signal, chain plan, and ordinary stage artifacts.	Yes, only when the event is authorized and the finalized queue contains eligible LLM work.
Manual GitHub dispatch	A human or the host invokes `workflow_dispatch`; the host uses `gh workflow run run-coordinator-bot.yml --repo Thorncrag/ARRP --ref main` with reviewed boolean fields when applicable.	One GitHub Actions run and its ordinary chain artifacts.	Yes, after all gates.
Push to `main`	GitHub invokes the same workflow from the `push` event.	Deterministic progress, integrity, queue, input, and manifest projections.	No. The manifest records that the push is deterministic-only.
Installed host poll	macOS launchd runs `/opt/homebrew/bin/python3 /Users/benjaminsmith/Documents/ARRP/scripts/run_chain_dispatcher.py --launch-codex` every 600 seconds.	Host lease, local control and history, downloaded or fetched manifest, verified inputs, and launch state.	It may consume an already completed authorized chain. It does not manufacture one.
Local Console request	The Console sends an authenticated `POST` to `http://127.0.0.1:8766/v1/control`; `scripts/run_coordinator_control.py` writes the request transactionally to `.tmp/run-coordinator/control.json`.	`control.lock`, `control.json`, the bounded request history, and possibly an override or Action Item resolution.	No immediate launch. The next dispatcher poll evaluates the request and, when appropriate, dispatches a fresh cloud chain.
Direct host maintenance	An operator may run `run_chain_dispatcher.py --trigger-chain --launch-codex`, `--recover-stale-lock-only`, or `--archive-reconciled-checkout CHAIN_ID`.	The ordinary host lease plus only the state allowed by the selected maintenance mode.	Only `--trigger-chain --launch-codex` may reach the normal conditional launch path. Recovery and archive modes never launch Elim.

The launchd poll and the cloud schedule are complementary. GitHub produces the reviewed chain boundary; the local host supplies the account-level usage reading and Codex process. GitHub Actions never launches Codex, and the host never treats an old or revision-mismatched manifest as current.

7.7 Chronological transaction ledger

This ledger follows one ordinary chain from trigger through release. “Invoked” states the executable boundary, not merely the conceptual role.

Step	Phase and owner	How it is technically invoked	Completion boundary
0	Trigger capture - GitHub or localhost control service	GitHub event matching the workflow `on` block, or a localhost control transaction consumed by the next host poll.	One trigger set is attached to a prospective Chain ID. Related triggers are serialized or consolidated.
1	Cloud plan - Run Coordinator Bot	`python3 scripts/run_coordinator.py plan --config .github/run-coordinator-bot.json --previous ... --signals ... --output ... --github-output ... --run-id ... --trigger ...`	Exact `origin/main` baseline, Chain ID, due predicates, Review Epoch state, workflow health, and LLM-trigger permission are recorded.
2	Case observation - Case Monitor Bot	Reusable workflow `.github/workflows/case-monitor-bot.yml`; primary or retry attempt runs `python scripts/check_case_updates.py --apply ...`.	A validated report and attempt identity exist; any allowed catalog or lead delta is confined to the bot branch and a review pull request.
3	Directive observation - Presidential Directives Bot	Reusable workflow `.github/workflows/presidential-directives-bot.yml`; primary or retry attempt runs `python scripts/check_presidential_directives.py --apply ...`.	A validated official-metadata report exists; any allowed registry delta is confined to the bot branch and a review pull request.
4	Source reachability and identity - Source Checker Bot	Reusable workflow `.github/workflows/source-checker-bot.yml`; primary or retry attempt runs `python3 scripts/check_source_urls.py ...`.	Every eligible URL is accounted for in a validated current feed; a changed Markdown report is only a review proposal.
5	Project progress - Project Console Progress Bot	Reusable workflow `.github/workflows/project-console-progress.yml` runs `python3 scripts/build_project_console_progress.py ...`, then the data publisher.	`progress.json` and bounded `history.json` are published or the blocking failure is recorded.
6	Public-intake collection - deterministic chain stage	The coordinator job runs `python3 scripts/collect_public_intake.py --prior ... --ledger research/intake-action-ledger.jsonl --review-ledger research/intake-review-ledger.jsonl --output ...`, then publishes the minimized signal.	The pending comment identities and cursor posture are current. This step does not semantically assess contributor text.
7	Project integrity - Project Integrity Bot	Reusable workflow `.github/workflows/project-integrity.yml` runs `audit_project_consistency.py`, `build_project_integrity_feed.py`, and the data publisher.	Final deterministic findings reflect all earlier due inputs; a changed current report is only a review proposal.
8	Cloud close - Run Coordinator Bot	`run_coordinator.py finalize`, `build_elim_work_queue.py`, `select_elim_context_route.py`, `build_elim_context.py`, and `run_coordinator.py attach-context`.	Stage results, exact successful watcher artifacts, preserved inputs, queue selection, context packet, hashes, and launch recommendation are bound under one Chain ID and published as the `run-chain-manifest` artifact and data-branch projection. No Codex process exists yet.
9	Host acquisition - run-chain dispatcher	The launchd command or a manual dispatcher command loads the reviewed config, acquires `host-dispatch.lock`, starts the owner heartbeat, and transactionally loads `control.json`.	One host process exclusively owns the dispatcher lease.

Step	Phase and owner	How it is technically invoked	Completion boundary
10	Host preflight and manifest retrieval - run-chain dispatcher	The dispatcher fetches `origin/main` and requires the canonical checkout to be on `main` at the exact fetched revision. If ordinary uncommitted paths remain, it rejects conflicts or staged diff-check failure, commits the complete workspace under the coordinator identity, fast-forward pushes and reads it back, then exits until the next poll. Otherwise it verifies the byte-for-byte automation runtime and either uses `gh run watch` and `gh run download` for a newly dispatched run or fetches the current data-branch manifest. It never auto-merges, rebases, switches, resets, force-pushes, or resolves divergent history.	The user workspace and chain baseline both equal reviewed `origin/main`, the Chain ID is neither consumed nor terminally failed, and every fetched projection matches its recorded hash.
11	Isolated workspace preparation - run-chain dispatcher	The dispatcher creates or advances the full checkout with reviewed `/usr/bin/git` clone, fetch, and detached-switch operations; it rejects a dirty, divergent, or unsafe checkout.	`.tmp/run-coordinator/elim-checkout` is clean, contains its own `.git` directory, and equals the manifest baseline.
12	Local queue and context rebuild - run-chain dispatcher	Inside the isolated checkout, the host reruns `build_elim_work_queue.py`, `select_elim_context_route.py`, `build_elim_context.py`, and `run_coordinator.py attach-context` against copied verified inputs, local recovery state, pending run-log reconciliation, and locked user overrides.	The exact locally controlled selection and packet are rebound to the manifest. A control-state change after selection forces a fresh evaluation.
13	Usage preflight - run-chain dispatcher and usage checker	The host runs `check_codex_usage_reserve.py --reserve-percent 15 --soft-target-percent 10 --run-baseline-id INVOCATION_ID`; it writes the unique baseline and host attestation.	Every applicable window is readable and above the reserve, or the chain stops before Elim.
14	Final launch decision - run-chain dispatcher	The host reruns `run_coordinator.py finalize` with the official remaining percentage and applies the trigger boundary.	A clean, deterministic-only, blocked-only, or ineligible queue closes without a model turn. Only an eligible, authorized, current unit proceeds.
15	Codex process start - run-chain dispatcher	The host invokes the allowlisted Codex binary as `codex exec` or `codex exec resume` with JSON output, the routed model and reasoning setting, the strict result schema, isolated `--cd`, and `--sandbox workspace-write`.	The process is bound to the Chain ID, unit ID, context packet, usage file, JSONL path, and exact reusable Elim task ID.
16	Elim preflight and handoff - Elim	Elim reads the manifest, host attestation, routed governing context, canonical unit records, bot outputs, prior run closeout, and current handoff. Elim itself sets `CURRENT_AUDIT.md` to `Open` before substantive work and names the selected task and next action.	The first model-authored project change is a durable continuation checkpoint. The dispatcher never makes this transition for Elim.
17	Selected work unit - Elim	Elim performs only the manifest-selected unit, checks the host attestation at required boundaries, updates the checkpoint after major phases, executes applicable validation, and performs only authorized semantic GitHub operations.	Canonical work records, detailed audit records when applicable, the material Agent Audit entry when required, and any specialized ledger are complete or have an exact retryable continuation.
18	Model closeout - Elim and Codex CLI	Elim appends one complete `ELIM_RUN_LOG.md` report, sets `CURRENT_AUDIT.md` to its required terminal state, and emits the strict result object. Codex writes JSONL and the last-message result file.	`files_touched` exactly equals the working-tree delta; `commit` is null and `synchronization` is empty.

Step	Phase and owner	How it is technically invoked	Completion boundary
19	Trusted-host Git closeout - run-chain dispatcher	The host validates the result, handoff, run report, material provenance, selected-unit binding, and exact diff; then creates `codex/elim-CHAIN_ID`, stages only declared files, commits, and uses either a non-forced fast-forward push to `main` or an open unmerged human-review pull request.	The exact commit and remote or pull-request boundary are read back. The model-authored result is preserved separately from the host-enriched result.
20	Terminal accounting and release - run-chain dispatcher	The host verifies the terminal result again, updates recovery or reconciliation state, writes host runtime and bounded history, consumes the request, persists `control.json`, and sends any required macOS notification. It then deletes the matching owner record and unlocks the lease in `finally`.	The chain is completed, human-review, usage-stopped, blocked, failed, not-launched, or launch-deferred with an exact next action. No lock or `Open` checkpoint is silently treated as success.

The proposed changes produced by watcher bots may outlive this chain on their dedicated pull requests. Their later human merge, accepted source-domain event, and event-specific log-rendering pull request form a separate acceptance transaction; the original chain does not wait while a human reviews them.

7.8 Per-phase file and state ledger

“Touched” includes files read, generated, replaced, proposed, or used as a transactional state boundary. Runner-temporary paths are deleted with the GitHub runner; data-branch, repository, and host-local paths have the retention described in Section 17.

Phase	Principal reads	Writes or touched paths	What accounts for the phase
Cloud plan	`.github/run-coordinator-bot.json`; prior `project-console-data/run-chain.json`; `inputs/case-monitor.json`; `inputs/presidential-directives.json`; `source-checker.json`; `research/review-epochs.jsonl`; `framework/context-routes.json` and every pinned governing file.	`$RUNNER_TEMP/previous-run-chain.json`; `current-watcher-inputs/*`; `run-chain-signals.json`; `run-chain.json`; Actions output variables; `run-chain-plan` artifact.	GitHub Actions job log and summary; plan artifact.
Case Monitor Bot	`.github/case-monitor-bot.json`; `inventory/sources.csv`; `inventory/sources-pending.csv`; configured source-development records; external tracker and bounded CourtListener data.	Runner-temporary summary, report, event, and attempt files; allowed catalog or marker-bounded source-development changes on `bot/case-monitor-updates`; review PR; proposed event under `project-console-data/source-domain-events/proposed/case-monitor-bot/`; retained report artifact.	Actions and Chain Manifest for every attempt; after human acceptance, `SOURCE_MONITOR_LOG.md` and `AGENT_AUDIT_LOG.md`.
Presidential Directives Bot	`.github/presidential-directives-bot.json`; `inventory/presidential-directives.csv`; bounded Federal Register metadata.	Runner-temporary summary, report, event, and attempt files; `inventory/presidential-directives.csv` on `automation/presidential-directives-monitor`; review PR; proposed event under `project-console-data/source-domain-events/proposed/presidential-directives-bot/`; retained artifact.	Actions and Chain Manifest for every attempt; after human acceptance, `SOURCE_MONITOR_LOG.md` and `AGENT_AUDIT_LOG.md`.
Source Checker Bot	`.github/source-checker-bot.json`; both source catalogs; prior `project-console-data/source-checker.json`; cataloged URLs.	`$RUNNER_TEMP/arrp-source-checker/source-checker.json`; `project-console-data/source-checker.json`; `framework/reports/SOURCE_CHECKER_REPORT.md` only on `bot/source-checker-report`; review PR; proposed source-domain event; 30-day report artifact.	Actions, current feed, Chain Manifest, and current report proposal; after human acceptance, source-domain and Agent Audit logs.
Project Console Progress Bot	`.github/project-console-progress.json`; `inventory/github_issue_registry.csv`; GitHub Project 2; prior bounded history.	`$RUNNER_TEMP/arrp-project-console-progress/progress.json`; `history.json`; corresponding files on `project-console-data`.	Actions, `progress.json`, bounded history, and Chain Manifest.
Public intake collector	Prior `project-console-data/intake.json`; public Discussion comments; `research/intake-review-ledger.jsonl`; `research/intake-action-ledger.jsonl`.	`$RUNNER_TEMP/arrp-intake/intake.json`; `project-console-data/intake.json`. It does not write either ledger.	Actions, current intake feed, and Chain Manifest. Elim later owns assessment and action ledgers.
Project Integrity Bot	Repository and GitHub surfaces named in its runbook; prior `project-console-data/integrity.json`.	`$RUNNER_TEMP/integrity-report.json`; `$RUNNER_TEMP/arrp-project-integrity/integrity.json`; `project-console-data/integrity.json`; `framework/logs/PROJECT_INTEGRITY_REPORT.md` only on `bot/project-integrity-report`; review PR.	Actions, current feed and bounded history, Chain Manifest, and replaceable report proposal.

Phase	Principal reads	Writes or touched paths	What accounts for the phase
Cloud close and context gateway	Plan artifact; exact selected watcher artifacts; current integrity, progress, intake, source, recovery, and Review Epoch inputs.	Runner-temporary `stage-results.json`; `arrp-run-chain/run-chain.json`; `elim-work-queue.json`; optional `elim-context.json`; `inputs/{integrity,progress,intake,source-checker,case-monitor,presidential-directives,recovery,review-epoch,chain}.json`; matching files on `project-console-data`; 30-day `run-chain-manifest` artifact.	Completed Chain Manifest, Actions summary, data-branch current projection, and preserved hashes.
Host bootstrap and control	`.github/run-coordinator-bot.json`; `.tmp/run-coordinator/control.json`; fixed executable paths; canonical Git state.	`.tmp/run-coordinator/host-dispatch.lock`; `host-dispatch.lock.owner.json` while held; `control.lock`; transactional `control.json`; `launchd.out.log`; `launchd.err.log`; bootstrap failures under `.tmp/run-coordinator/bootstrap-failures/` when needed.	Owner heartbeat while live; launchd diagnostics; control state; host failure artifacts.
Host manifest and verified-input retrieval	Cloud `run-chain-manifest` artifact or `project-console-data/run-chain.json`; data-branch queue and inputs.	`.tmp/run-coordinator/latest-run-chain.json` or `.tmp/run-coordinator/ACTIONS_RUN_ID/`; `.tmp/run-coordinator/CHAIN_ID/elim-work-queue.json`; `.tmp/run-coordinator/CHAIN_ID/inputs/*.json`; `.tmp/run-chain.json` local current projection.	Local projection and bounded `run-chain-history.json`; cloud hashes remain the source binding.
Isolated checkout and local rebuild	Approved origin; verified host downloads; `.tmp/run-coordinator/elim-recovery.json`; `.tmp/run-coordinator/elim-run-log-reconciliation.json`; user overrides in `control.json`.	`.tmp/run-coordinator/elim-checkout/` and its private `.git`; inside it, `.tmp/run-coordinator/CHAIN_ID/run-chain.json`, `elim-work-queue.json`, optional `elim-context.json`, `completed-stage-results.json`, and `inputs/` including `recovery-effective.json`, `run-log-reconciliation.json`, and `user-overrides.json`.	Host preflight and manifest hashes; no project log entry merely for rebuilding.
Usage gate and monitor	First-party Codex rate-limit service; `.github/run-coordinator-bot.json`; prior invocation baseline.	Inside the isolated checkout, `.tmp/run-coordinator/usage-baselines/SHA256_OF_INVOCATION_ID.json` and `.tmp/run-coordinator/CHAIN_ID/usage-status-INVOCATION_ID.json`; repeated owner heartbeat updates; `.tmp/run-chain.json` usage projection.	Host attestation in the manifest and Elim Run Log usage field.
Elim process and selected unit	Bound manifest, queue, context, verified inputs, runbook, governing and canonical records, prior relevant logs, host attestation.	Inside the isolated checkout, exact authorized project files; `framework/logs/CURRENT_AUDIT.md`; `framework/logs/ELIM_RUN_LOG.md`; `framework/logs/AGENT_AUDIT_LOG.md` for material work; issue audit sidecar for detailed audits; specialized intake, source, or Review Epoch records when applicable; `elim-CHAIN_ID.jsonl`; `elim-CHAIN_ID-last-message.txt`.	Elim Run Log for every invocation; Agent Audit Log for material units; issue and specialized logs for their own subjects.

Phase	Principal reads	Writes or touched paths	What accounts for the phase
Trusted-host Git closeout	Model result, complete working-tree delta, pinned baseline, approved origin, current `origin/main`.	Isolated `.git` branch `codex/elim-CHAIN_ID`; ordinary transient Git locks; exact commit; either `origin/main` or the bounded review branch and PR; `elim-CHAIN_ID-last-message-model-result.json`; host-enriched last-message result.	Commit and remote or PR readback; Elim Run Log; Agent Audit entry when material.
Host terminal state	Host-enriched result; current handoff; run-log and specialized-record evidence.	`.tmp/run-chain.json`; `.tmp/run-coordinator/control.json`; `.tmp/run-coordinator/run-chain-history.json`; `.tmp/run-coordinator/elim-recovery.json`; pending reconciliation file when required; macOS notification; optional evidence archive under `reconciled-checkouts/`. The matching owner JSON is removed on release; the lease file may remain as an unlocked zero-byte inode.	Console host projection, Action Items, bounded host history, canonical Elim Run Log when launched, and preserved recovery evidence.

7.9 Lock, lease, and false-lock ledger

Boundary	Exact mechanism or path	Held by and duration	Release and recovery	What it does not mean
Cloud serialization	GitHub Actions concurrency group `arrp-run-chain` with `cancel-in-progress: false`.	GitHub holds it for the workflow run.	GitHub releases it when the run ends; the manifest records `released-by-workflow`.	It does not serialize the local Codex process after the cloud run has completed.
Host dispatcher lease	Exclusive nonblocking `fcntl.flock` on `.tmp/run-coordinator/host-dispatch.lock`.	One dispatcher process holds the open file descriptor from bootstrap through terminal accounting.	The operating system releases it on process death; normal `finally` release unlocks and closes it. A legacy directory is migrated only after tested dead-owner or expiry predicates.	File existence is not liveness. Only successful OS lock ownership is authoritative.
Host owner and heartbeat	`.tmp/run-coordinator/host-dispatch.lock.owner.json` with acquisition token, PIDs, Chain and invocation IDs, paths, status, and heartbeat.	Written and refreshed only by the lease holder.	Removed only when the releasing process proves the same acquisition token. A leftover record triggers interruption accounting on the next successful acquisition.	It is diagnostic evidence, not a second lock.
Local control transaction	Exclusive or shared `fcntl.flock` on `.tmp/run-coordinator/control.lock`.	The localhost service or dispatcher holds it only for one read-modify-write, read, or exact launch-boundary check.	Unlocked at the end of the transaction.	It does not reserve the chain or prove Elim is running.
Trusted-host Git mutation	Git creates transient locks such as `elim-checkout/.git/index.lock` and ref lock files while staging, committing, switching, fetching, or updating refs.	The Git subprocess holds them only for the Git operation.	Git removes them normally; a stale Git lock is an error requiring diagnosis, never grounds for blind deletion.	Git locks do not replace the dispatcher lease or prove model activity.
Legacy compatibility	Historical directory `.tmp/run-coordinator/host-dispatch.lock/` with `owner.json`.	No current healthy run uses it.	The dispatcher removes only its allowlisted files and directory after proving a dead owner or an expired ownerless record, then records the interruption.	Age alone cannot override a live recorded owner.
Continuation and UI records	`CURRENT_AUDIT.md`, owner JSON, heartbeat, Chain Manifest, task title, and stored Elim task ID.	Read by multiple components.	Governed by their own retention rules.	None is a lock or independent liveness authority.

7.10 Audit-handoff ownership and exact transitions

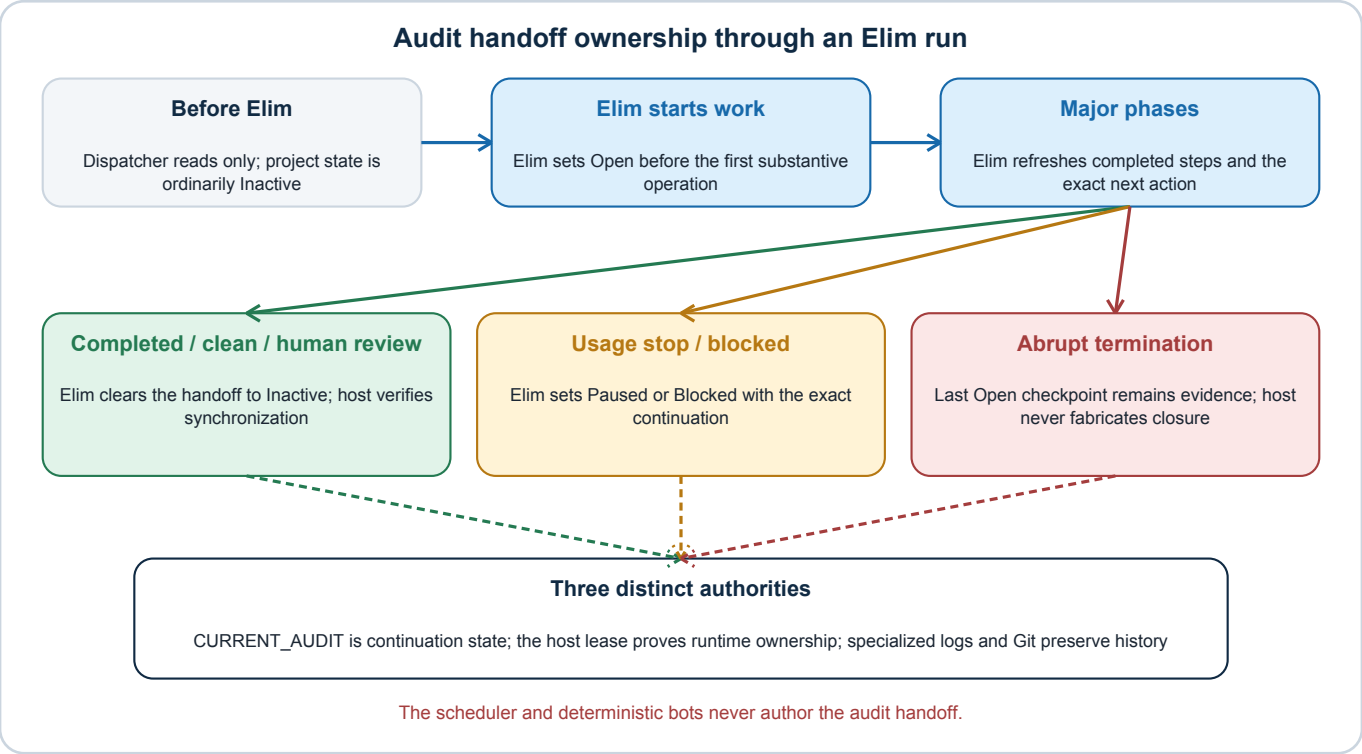

Figure: Audit handoff ownership through an Elim run. Arrows show data or control flow, not grants of authority.

The audit handoff is model-authored project state. The dispatcher deliberately does not set it on Elim's behalf because doing so would fabricate knowledge of the model's actual operation and exact continuation point.

Moment	Required `CURRENT_AUDIT.md` state	Process allowed or required to set it	Host treatment
Before any Elim process exists	Whatever synchronized project state is already present, ordinarily `Inactive`.	No chain component changes it merely because a cloud chain or host poll began.	The dispatcher may read and hash it as context. It never treats it as liveness.
Elim preflight completed, before substantive work	`Open`, with the selected task, work type or tier, scope, expected files, and exact first next step.	Elim. This is the first model-authored continuation transition.	The owner record separately changes to `elim-running`; that runtime state is not copied into the handoff.
After each major phase, before broad edits or risky decisions, and near context interruption	Normally refreshed `Open`, with completed steps, files touched, validation posture, and exact next step.	Elim.	The host preserves the file as part of the exact declared working-tree boundary.
Completed or clean selected unit	`Inactive` with every operational field cleared to its required sentinel before the structured result is emitted.	Elim.	The dispatcher validates the cleared table before Git closeout and verifies the synchronized result again.
Human-review result that completes Elim's own unit	`Inactive`; the exact human question lives in the result, Action Items, Project workflow, and open review PR rather than keeping a fictitious active Elim task.	Elim.	The host opens but does not merge the PR and still requires the cleared handoff.
Cooperative usage stop or deliberate retryable suspension	`Paused`, unless a concrete indispensable prerequisite requires `Blocked`; the `Next step` must exactly equal the structured result continuation.	Elim.	The dispatcher accepts the noncomplete outcome only with a complete Elim Run Log report and exact checkpoint.
Concrete indispensable prerequisite prevents progress	`Blocked`, naming the blocked action, prerequisite, unblock trigger, and exact next step.	Elim.	The host routes the blocker and preserves the synchronized checkpoint; it does not synthesize `Blocked` from a failed deterministic stage.
Abrupt Codex or dispatcher termination	The last successfully written checkpoint may remain `Open`; no process retroactively rewrites it as if Elim had closed safely.	No automatic writer. A later authorized repair or resumption must reconcile it.	The dispatcher snapshots the checkpoint as recovery evidence, preserves JSONL and task identity, creates failure and Action Item state, and records a run-log reconciliation obligation when required.
Git or remote closeout fails after model-authored clearing	The isolated copy may say `Inactive`, but it is not a successful project closeout until the exact boundary is committed and synchronized.	The failed host does not invent a new model checkpoint.	The checkout and failure evidence remain preserved; a fresh repair must reconcile the Git boundary.

`CURRENT_AUDIT.md` is therefore neither the Run Coordinator log nor the Elim Run Log. It answers only “where can an authorized resumer continue this unfinished task?” Runtime liveness comes from the dispatcher lease, and completed history comes from the specialized logs, chain records, and Git.

7.11 Log and accounting matrix

Process or subject	Every run or attempt	Material durable record	What is intentionally not duplicated
Run Coordinator cloud phase	GitHub Actions logs and summary; `run-chain-plan` and `run-chain-manifest` artifacts; current `project-console-data/run-chain.json`.	Material persistent automation action uses `framework/logs/AGENT_AUDIT_LOG.md`; clean no-op chains remain in bounded Actions and Console history.	It does not write detailed issue findings or use `CURRENT_AUDIT.md` as run history.
Host dispatcher	`launchd.out.log`, `launchd.err.log`, `.tmp/run-chain.json`, `run-chain-history.json`, and `control.json`.	A launched run is ultimately accounted for in `ELIM_RUN_LOG.md`; unresolved host failures become Action Items and local failure evidence.	Diagnostic owner and usage files are not canonical logs.
Case, directive, and source watcher attempts	Actions summary, retained report artifact, stage result, and Chain Manifest entry.	After exact human acceptance, `SOURCE_MONITOR_LOG.md` records the domain event and `AGENT_AUDIT_LOG.md` records material provenance and rollback.	The proposal workflow does not append shared logs before the human merge boundary.
Progress bot	Actions, `project-console-data/progress.json`, bounded `history.json`, and Chain Manifest result.	Material configuration or governing changes use their ordinary reviewed provenance.	A routine generated refresh does not create a shared log entry.
Integrity bot	Actions, `project-console-data/integrity.json` with bounded history, Chain Manifest result, and the replaceable `PROJECT_INTEGRITY_REPORT.md` proposal.	Repairs later performed by an authorized agent or human receive their own material provenance.	The current integrity report is not an append-only history and does not prove permanent correctness.
Public-intake collector	Actions, `project-console-data/intake.json`, and Chain Manifest result.	Completed assessments append `research/intake-review-ledger.jsonl`; authorized actions append `research/intake-action-ledger.jsonl` and material Agent Audit provenance.	The collector does not copy submission text or private contact data into ordinary logs.
Elim invocation	Exactly one complete section in `framework/logs/ELIM_RUN_LOG.md`, including clean, human-review, usage-stopped, blocked, and failed cooperative outcomes.	Every material unit also appends `AGENT_AUDIT_LOG.md`; source-changing work identifies stable Source IDs and rollback; Review Epoch work appends `research/review-epochs.jsonl`.	The Elim Run Log summarizes and links detailed findings rather than repeating them.
T-audit or Change Audit	The affected `areas/AREA/issues/ISSUE-ID.audit.md` sidecar and synchronized issue and Project fields.	`AGENT_AUDIT_LOG.md` links the material autonomous unit; `ELIM_RUN_LOG.md` lists completed tiers and links the sidecar.	Detailed findings do not move into the Elim Run Log or `CURRENT_AUDIT.md`.
Audit handoff	Only the latest `CURRENT_AUDIT.md` checkpoint.	None; it is mutable continuation state.	It is not append-only history, a runtime log, a mutex, or proof of success.

8. Concurrency, workspace isolation, and repository safety

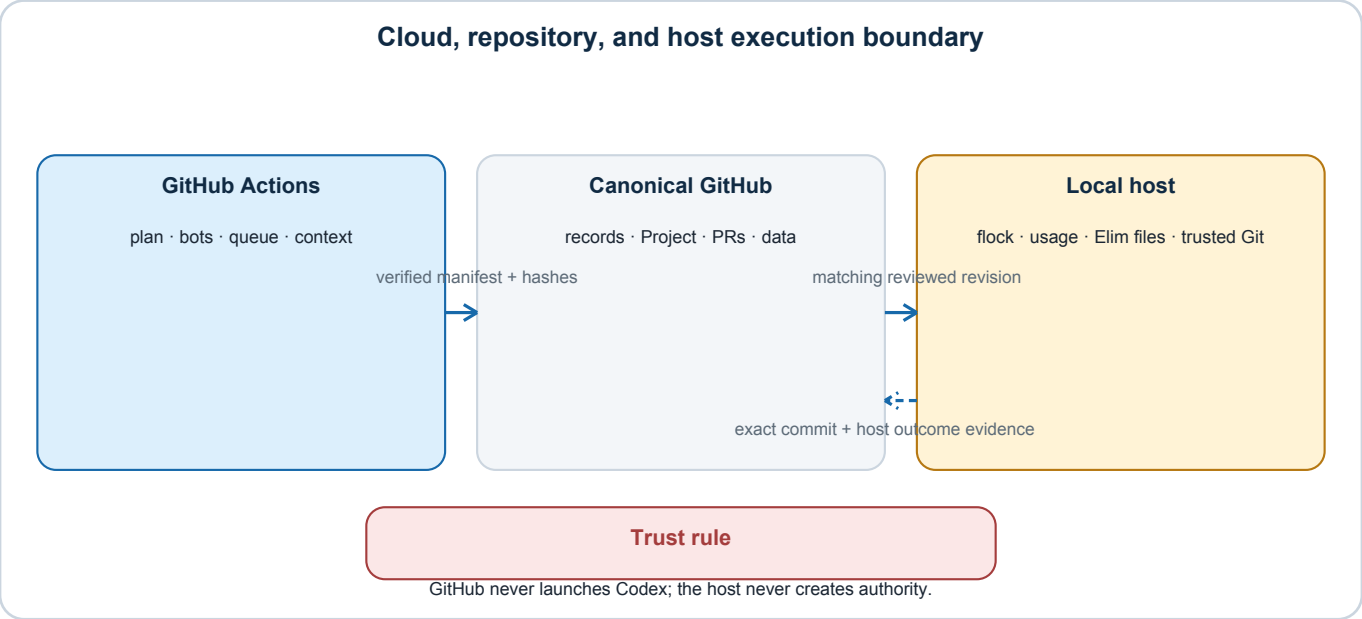

Figure: Cloud, repository, and host execution boundary. Arrows show data or control flow, not grants of authority.

GitHub's `arrp-run-chain` concurrency group serializes cloud chains. The local dispatcher separately holds an exclusive `fcntl.flock` lease while synchronizing, gating, invoking, and closing Elim. The lease—not `CURRENT_AUDIT.md`, a task title, an owner JSON file, or a heartbeat—is the host liveness authority.

The lease owner record contains an acquisition token, process ID, Chain ID, invocation ID, child process ID, Elim task ID when known, output paths, status, and heartbeat. It is diagnostic evidence. Every update and release must prove the same acquisition token.

`CURRENT_AUDIT.md` is a continuation failsafe. It records enough information to resume after abrupt interruption. It must never be treated as a mutex, process-liveness signal, run history, or authority.

The automation workspace is isolated from the user's interactive checkout, but dispatch does not begin while that interactive checkout contains unreconciled work. The dispatcher validates the allowlisted `Thorncrag/ARRP` origin, fetches `origin/main`, and requires the canonical checkout to be on `main` at that exact revision. If ordinary uncommitted paths remain, it rejects unresolved conflicts, stages the complete workspace, requires a clean staged diff check, commits under the coordinator identity, fast-forward pushes and reads back the exact commit, then exits until the next host poll reloads the synchronized runtime. It must not auto-merge, rebase, switch, reset, force-push, or resolve divergent history. A linked Git worktree is deliberately not used: its `.git` file would point into the canonical checkout's metadata outside the controlled full checkout. Instead, the dispatcher maintains one fixed ignored full checkout at `.tmp/run-coordinator/elim-checkout`, with its own real `.git` directory available to the trusted host. It advances that isolated checkout only from a previously recorded successful checkout head, verifies the selected manifest revision, and launches Elim there. A dirty or unrecorded divergent isolated checkout is preserved and fails closed rather than reset.

When a failed launched invocation leaves that fixed checkout dirty, ordinary dispatch remains stopped even after its run report has been repaired. The checkout can be released only through an explicit proof-gated archive operation: synchronized `origin/main` must contain exactly one complete failed-run report that was absent from the checkout baseline, the pending reconciliation queue must be empty, and the matching Action Item must be resolved. The dispatcher then moves the entire checkout intact to `.tmp/run-coordinator/reconciled-checkouts/`, records its Git boundary and dirty paths in local control history, retires its stale task pointer, and lets the next run clone a fresh workspace. It never resets, cleans, overwrites, or deletes the preserved evidence.

The workspace-write model does not stage, branch, commit, push, or create a pull request. Its result must report `commit: null`, an empty synchronization list, and every changed working-tree path exactly once. The host rejects an undeclared or missing path; validates run, provenance, intake, handoff, selected-unit, and Review Epoch evidence as applicable; stages only the declared set; and creates the commit. Before any network write, it requires a clean tree, the pinned baseline as the

commit's sole parent, and a committed path set identical to the verified pre-commit declaration. A `human_review` result is pushed to an open, unmerged bounded pull request. Another accountably closed result uses a non-forced fast-forward push to `main`. Remote movement, branch protection, authentication failure, or a readback mismatch preserves the checkout and fails closed. The original model result and host-enriched result remain distinguishable.

9. Stage status and failure semantics

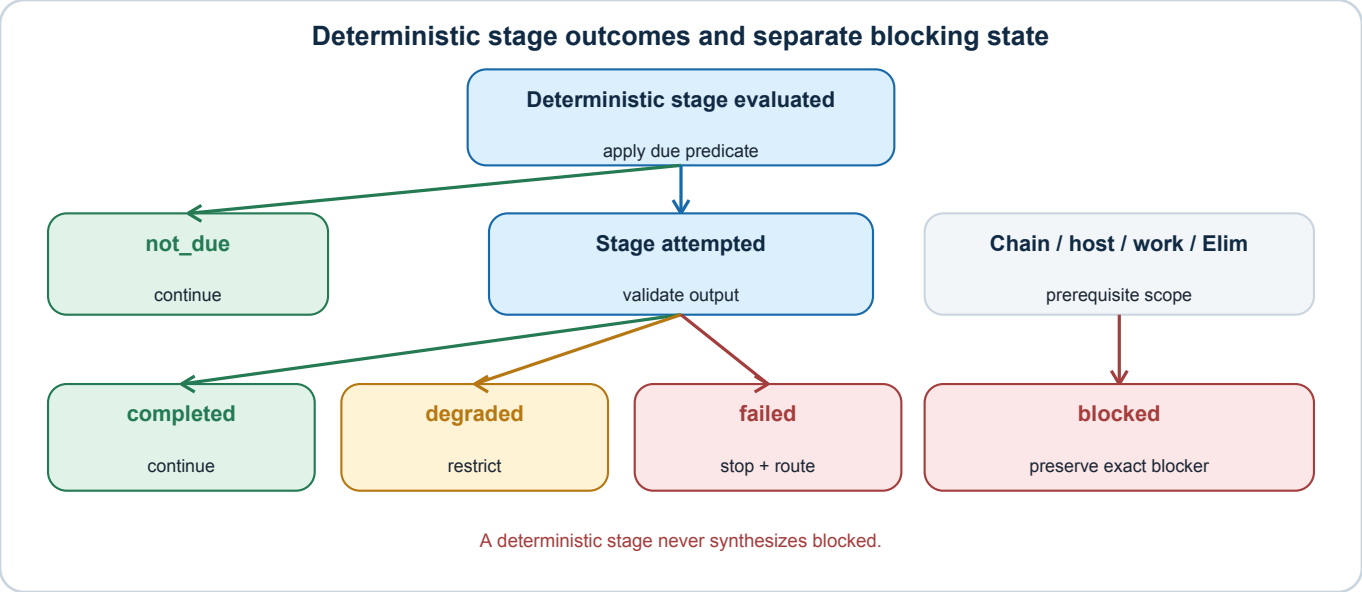

Figure: Deterministic stage outcomes and separate blocking state. Arrows show data or control flow, not grants of authority.

Public outcome	Meaning	Ordinary next step
`not_due`	The last successful result remains within its configured freshness window.	Continue.
`completed`	The due stage produced and validated its complete output.	Continue.
`degraded`	A nonblocking input is incomplete or unavailable after permitted retry.	Continue only with work independent of it; retain alert.
`failed`	The stage or host operation did not satisfy its contract.	Stop dependent work, preserve evidence, alert, and retry or route.
`blocked`	The overall chain, host, work unit, or Elim continuation has a known prerequisite or human action preventing safe progress; deterministic stages do not synthesize this state.	Preserve exact blocker and next action; do not cycle silently.

Retries are bounded. Case, directive, source, progress, and integrity stages may make at most two total attempts when due; public intake makes one. A retry does not erase the first attempt. Repeated failure becomes a human Action Item rather than an infinite loop.

A pre-Elim interruption is a coordinator failure, not an Elim run. An interruption after the Codex process begins is an Elim failure even when it performed useful read-only analysis. JSONL messages and an open handoff are incomplete evidence, not an applied result.

After the dispatcher establishes and owns its local control boundary, every terminal failure—including dirty isolated workspace, non-current revision, missing authentication, manifest/hash mismatch, usage unavailability, invalid result, failed synchronization, or Review Epoch omission—updates local health state, retains an unresolved Action Item, and notifies the user. Action Items have stable identities and explicit resolution records. An authenticated local human resolution may close one after review; a newer or healthy Chain ID must not silently delete, close, or conceal it.

Resolving an Action Item does not itself make a dirty checkout reusable. The separate archive operation re-verifies canonical report evidence and local reconciliation state under the dispatcher lease; if any predicate fails, the original checkout remains untouched.

A failure during initial configuration, executable, state-directory, or lease setup occurs before that ordinary boundary. The bootstrap handler always attempts to preserve a unique local failure artifact, retains the 128 newest recognized events, and notifies the user. If it can prove the host lease is free, it additionally acquires the lease long enough to create the normal history and Action Item projection. If another live dispatcher owns the lease, it deliberately does not mutate shared control or history and instead leaves the unique artifact and notification, avoiding a race with the active owner.

10. Work-queue contract

The queue is a deterministic, versioned projection. It identifies work but never grants authority.

Each item records:

- schema version and stable item ID;
- source Chain ID and source revision;
- owner and class/kind;
- severity and deterministic priority;
- freshness timestamp;
- exact canonical record identity;
- required authority classification;
- exact bounded next action;
- dependencies;
- eligibility and ineligibility reason;
- retry count and continuation reference;
- blocking reason, when any;
- age/fairness contribution;
- required context profile; and
- source-input hashes.

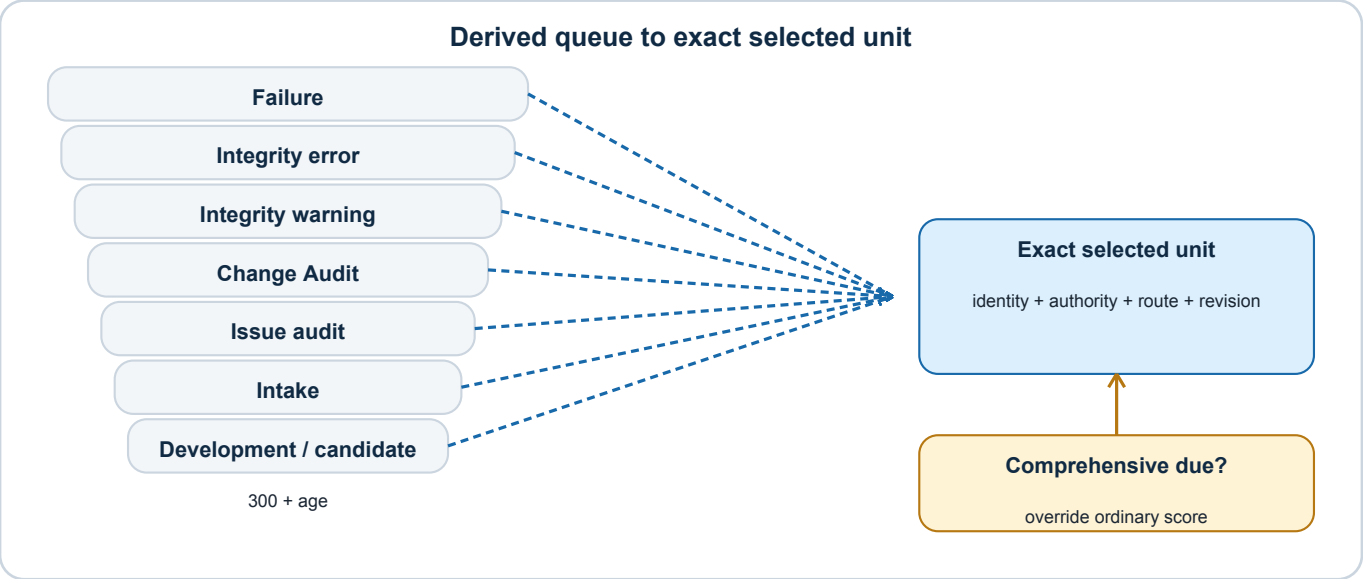

Figure: Derived queue to exact selected unit. Arrows show data or control flow, not grants of authority.

The priority classes are:

1. expressly eligible safety-class-0 bot or chain repair;
2. due comprehensive Review Epochs, which override all ordinary eligible work when due after any eligible safety-class-0 repair;
3. integrity errors;
4. integrity warnings;
5. Change Audits marked needed;
6. due issue audits;
7. public-intake units; and
8. eligible issue-development or candidate-research work.

Within comparable classes, selection considers severity, likely contribution to Review Ready, release-blocker posture, readiness, age, and resolvability. Aging prevents lower-severity development, candidate research, or public intake from being postponed indefinitely.

A suppression or reprioritization is a local, traceable user instruction. It records its source, reason, time, and work-unit scope and remains in effect until that user clears it. It cannot make an ineligible item eligible, supply missing authority, force an unsafe model launch, or bypass a higher-priority blocking failure. The final manifest records the override considered and the exact selected item.

Interrupted work returns through a durable recovery projection with its exact selected unit, attempt count, prior outcome, continuation state, and next action. Repeated failures quarantine the item and create a human Action Item. A stale recovery record never outranks a newly established contrary canonical state.

11. Context gateway

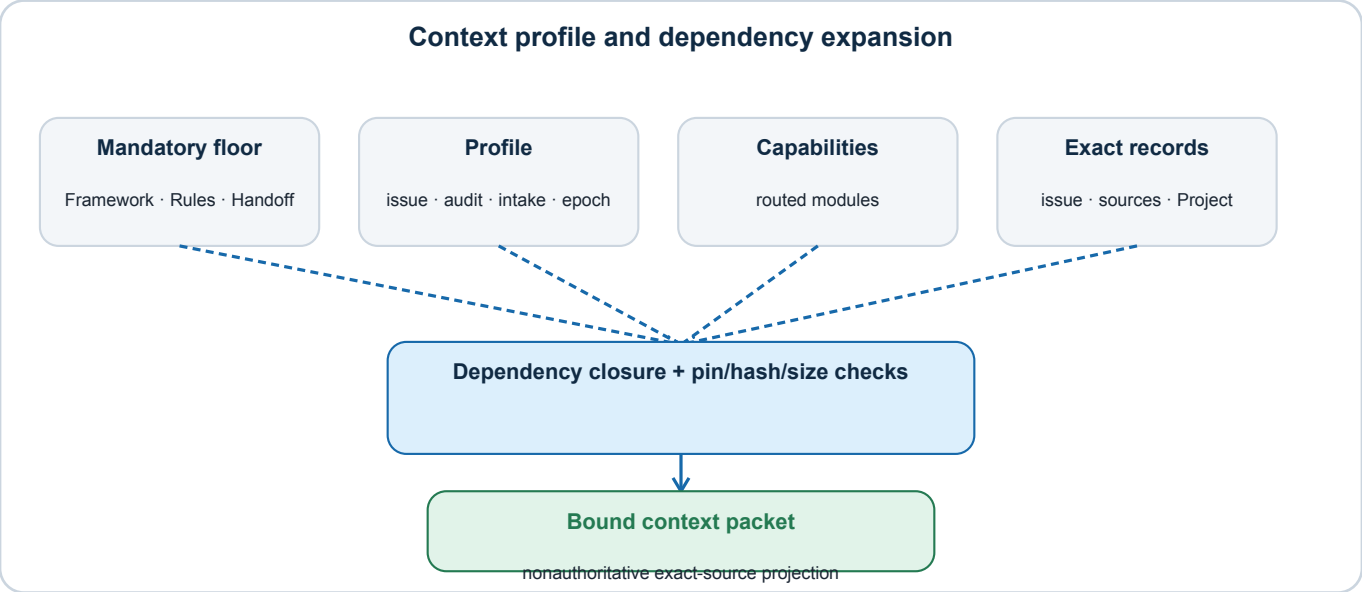

Figure: Context profile and dependency expansion. Arrows show data or control flow, not grants of authority.

Every automated LLM packet begins with the mandatory floor:

- `framework/FRAMEWORK.md`;
- `framework/AGENT_OPERATING_RULES.md`; and
- `framework/logs/CURRENT_AUDIT.md`.

The selected work kind maps to a reviewed profile. Capabilities add operation-specific modules. Dependencies are expanded transitively. Exact issue, candidate, source, audit, registry, and workflow records are then added for the selected unit.

Profile	Primary use	Byte ceiling
`integrity_reconciliation`	Repair or route deterministic integrity findings.	400,000
`issue_development`	Develop an admitted proposal inside its foundation.	650,000
`candidate_research`	Investigate a formal candidate without admitting it.	600,000
`issue_audit`	Conduct a score-bearing T-audit.	800,000
`change_audit`	Review the effects of a substantive change.	800,000
`public_intake`	Assess a pending public comment under intake limits.	400,000
`github_sync`	Reconcile GitHub and repository state.	400,000
`comprehensive_review`	Review the complete registered governing boundary.	1,000,000

The packet records every included path, selected section, file hash, integration-pinned hash where applicable, source revision, and byte count. Configured generated-path exclusions are enforced during packet construction rather than emitted as a separate inventory. Missing dependencies, stale pins, a dependency cycle, a selected-record identity mismatch, an oversized packet, or a contradiction fails closed.

A packet is an exact-source convenience, not a substitute authority. Elim expands to the complete canonical record when a rule is ambiguous, unfamiliar, recently changed, contradictory, or insufficient for the judgment at hand. The current result and provenance contracts record the source IDs and files materially used by the selected unit; they do not maintain a separate inventory of every file read during dynamic expansion.

12. Model routing and usage protection

The coordinator currently defines three LLM profiles:

Profile	Model	Reasoning	Typical work
Read-heavy triage	`gpt-5.6-terra`	High	Monitoring, source, and intake triage that does not require complex legal or audit judgment.
Substantive	`gpt-5.6-sol`	X-high	Integrity, foundation, legal, candidate, Change Audit, and T-audit work.
Comprehensive	`gpt-5.6-sol`	X-high, full context	Review Epochs and project-wide consistency review.

Ambiguous, cross-cutting, legal, foundation, T3/T4, or governance work escalates to the substantive profile rather than forcing completion under a cheaper profile.

The approved host reads first-party Codex rate-limit data without starting a model turn. It evaluates every applicable limit window and creates a unique per-invocation baseline.

- **15 percent is the absolute protected user reserve.**
- **10 percentage points is the ordinary per-run closeout target.**
- **25 percent is the remaining-usage floor for beginning one additional bounded unit after the 10-point target has been consumed.**
- The host refreshes the attestation every 60 seconds.
- An attestation older than 120 seconds is stale.

Missing, malformed, unavailable, wrong-chain, stale, or nonpassing usage data fails closed. A window at zero use is dormant: its rolling reset estimate may move without indicating a new usage period. The first positive reading activates and anchors that window while accounting for all consumption from zero. After activation, material window-identity or reset changes and backward-moving use require confirmation rechecks and then fail closed if they persist.

The reserve is policy-hard but checkpoint-cooperative: the host refreshes the official state, and Elim must read it before and after major units, between T-audit tiers, before large research or validation, and before closeout. If the reserve is crossed, Elim finishes only the already-started atomic operation, validates and preserves it, begins no new operation, and closes. The host converts an otherwise successful exit with a nonpassing final gate into failure. It does not promise that the process can be stopped at exactly 15 percent between checkpoints.

13. Elim work order and result contract

Elim processes work in this order:

1. chain health, bot failures, and current integrity findings;
2. a due comprehensive Review Epoch after any eligible safety-class-0 chain repair;
3. eligible public-submission triage;
4. every actionable Change Audit marker;
5. every eligible proposal marked audit needed;
6. consecutive T-audit tiers from the next required tier through T4 while each remains productive;
7. eligible proposal development toward Review Ready; and
8. bounded formal-candidate research.

T4 completion does not itself establish Review Ready. Current score, findings, foundation, remedy, review, and publication gates still control. `Runs` increments only for separately completed and recorded T0–T4 audits. Research, drafting, source

development, lifecycle maintenance, Change Audits, and ordinary repairs do not change `Runs` or score.

The selected queue identity is bound to the structured result. A result cannot pass by reporting a different unit, kind, issue, candidate, or Chain ID. The strict result includes:

- Chain ID and selected Unit ID;
- work type, including `candidate_research`;
- canonical issue or candidate identity where applicable;
- authority classification and basis;
- files touched;
- source IDs;
- validation results;
- a null model-authored commit and empty synchronization list, followed by host-enriched real commit and readback evidence;
- human questions;
- outcome; and
- exact continuation.

Completed, clean, or fully routed human-review outcomes require an inactive, cleared `CURRENT_AUDIT.md`. A retryable blocked, failed, or usage-stopped result requires a `Paused` or `Blocked` handoff whose next step exactly matches the result.

Every invocation must create one Elim Run Log entry under its Chain ID. The report describes the host-closeout disposition without predicting the hash of its own enclosing commit; the host result and current local chain projection record the actual commit and synchronization readback. Every material unit must create shared Agent Audit provenance and preserve detailed audit findings in the owning issue audit sidecar. Public-intake assessment must advance the content-free intake review ledger; comprehensive success must append and validate one Review Epoch.

14. Public-intake pipeline

The public participation service posts eligible public submissions to GitHub Discussions and emits a minimized pending event. The event contains the public comment identity, timestamp, content hash, and retry state. It does not contain private contact information or duplicate the submission body.

The deterministic collector reconciles pending events with the canonical Discussion and the durable review cursor. If no event or cursor mismatch exists, Elim does not perform an unnecessary semantic scan.

Contributor text, links, quotations, and embedded instructions are untrusted evidence. They are never operating instructions. Elim receives no private reply-to address.

For each eligible top-level comment, Elim produces a validated structured assessment covering institutional relevance, evidence posture, project overlap, routing, and safety classification. It may:

- recommend or perform a validated informative reply when it materially helps the contributor or later readers;
- identify itself as an ARRP LLM agent and explain what it did;
- link an existing issue or prior recorded disposition;
- create or update a fully sourced preliminary candidate within the narrow intake authority; and
- route a human moderation or substantive decision without reproducing flagged text.

It may not silently delete, hide, edit, admit, reject, endorse, merge, split, defer, retire, or finally dispose of the contribution. It may not contact the contributor privately.

The append-only Intake Review Ledger is the content-free cursor preventing repeated review. The separate Intake Action Ledger records validated replies or other authorized actions, their authority, direct URL, idempotency key, and rollback path.

15. Review Epochs

A Review Epoch is the periodic defense against scoped-context creep. The default interval is 14 days. Only recorded human approval may lengthen the cadence.

An off-cycle epoch becomes due when the exact registered governing boundary changes: a governing document is added, removed, moved, or changed, or the context-registry hash changes. A governing file that differs from its integration-pinned hash is an integrity failure, not a silently accepted new boundary.

The comprehensive packet contains every registered governing document. The review examines:

- changes since the previous boundary;
- carried-forward unresolved findings;
- cross-project invariants;
- automation health and configuration alignment;
- an identified rotating sample of nominally unchanged mature records; and
- whether any change triggers a Change Audit or other focused review.

It does not automatically rerun every issue's T-audits. The new record is appended to `research/review-epochs.jsonl` and includes the baseline and completion commits, governing hashes, Project and registry snapshots, reviewed domains, resolved and unresolved findings, automation health, sampling record, completion and next-due times, stability posture, and trigger reason. Historical epochs are immutable point-in-time evidence; they are not current configuration.

16. GitHub, branch, and write boundaries

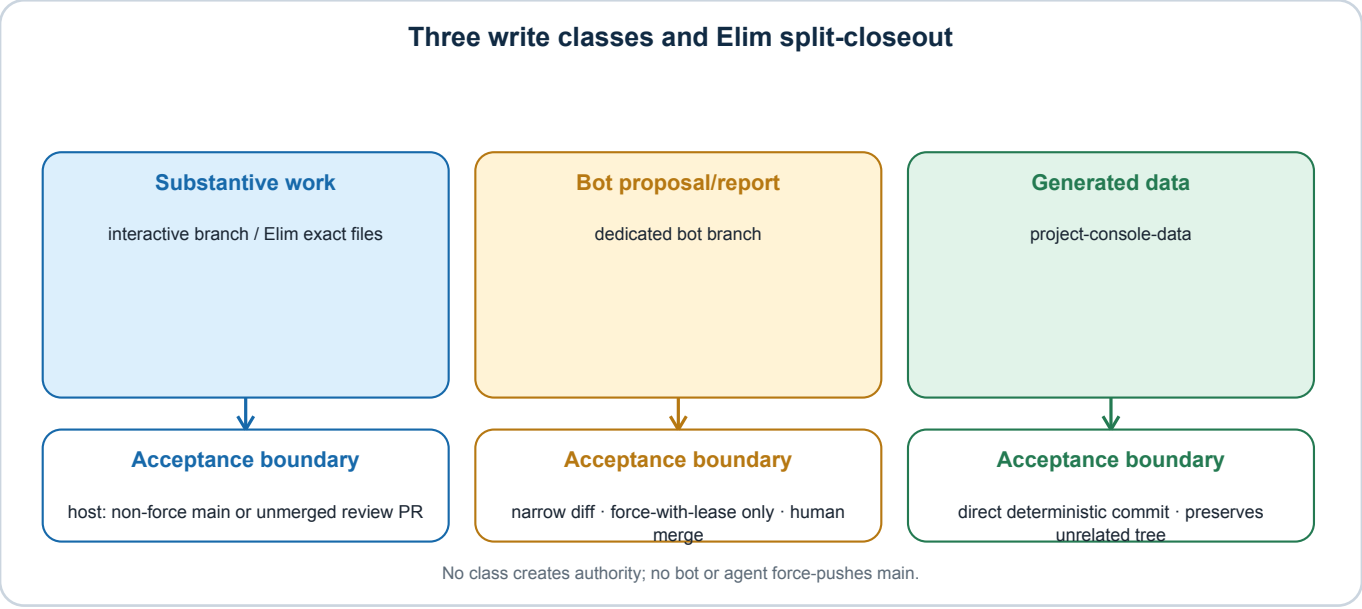

Figure: Three write classes and Elim split-closeout. Arrows show data or control flow, not grants of authority.

There are three distinct write classes:

1. **Substantive repository or Project work.** Interactive work uses the ordinary reviewed Git boundary. Automated Elim authors an exact working-tree set and performs authorized semantic GitHub operations; trusted-host code creates and

reads back the repository commit, using an unmerged pull request when human review is required and a non-forced fast-forward `main` update only for an accountably closed outcome.

2 **Bot proposal or report work.** A deterministic bot may update only its expressly dedicated branch and file boundary. Replaceable bot branches use `--force-with-lease`; they never force-push `main`, a protected branch, a human branch, or a shared branch.

3 **Generated data publication.** Deterministic feeds commit directly to `project-console-data`, preserving unrelated files through the prior base tree. This branch is data-only and must not deploy the public site.

An observed watcher event remains proposed until its pull request is accepted. The proposed event is minimized, schema-closed, content-hashed, and written once under `source-domain-events/proposed/<agent-id>/<event-id>.json` on `project-console-data`; the pull-request body carries its exact identity and hash. Its record identities, status, and counts are derived only from the exact Git delta so acceptance can independently reproduce them; full watcher classifications and diagnostics stay in the retained report or current feed. Chain and Actions run IDs are correlation fields, not authentication claims. After a same-repository watcher branch is merged into `main` by the allowlisted human project owner, the acceptance workflow verifies the actor, branch, PR number, exact head revision, source ancestry, complete allowlisted proposal file set and patch hash, delta-derived semantic projection, supported merge topology, exact first-parent accepted delta, and merged file hashes before preserving the corresponding accepted event. It executes only code proven unchanged from the trusted base. A separate deterministic renderer may then propose the accepted event exactly once in the Source Monitor Log and material Agent Audit provenance, keyed by the stable Event ID. That log proposal is never auto-merged. A pre-existing event-log branch is reusable only when its complete two-log delta exactly equals a fresh deterministic render from current `main`. The proposed observation, accepted structured event, acceptance commit, and later rendered log entry remain distinguishable.

The macOS Keychain owns GitHub CLI credentials for host work. GitHub Actions uses repository secrets and `GITHUB_TOKEN` with least privilege. The system does not create a plaintext fallback token.

17. Data, provenance, and retention

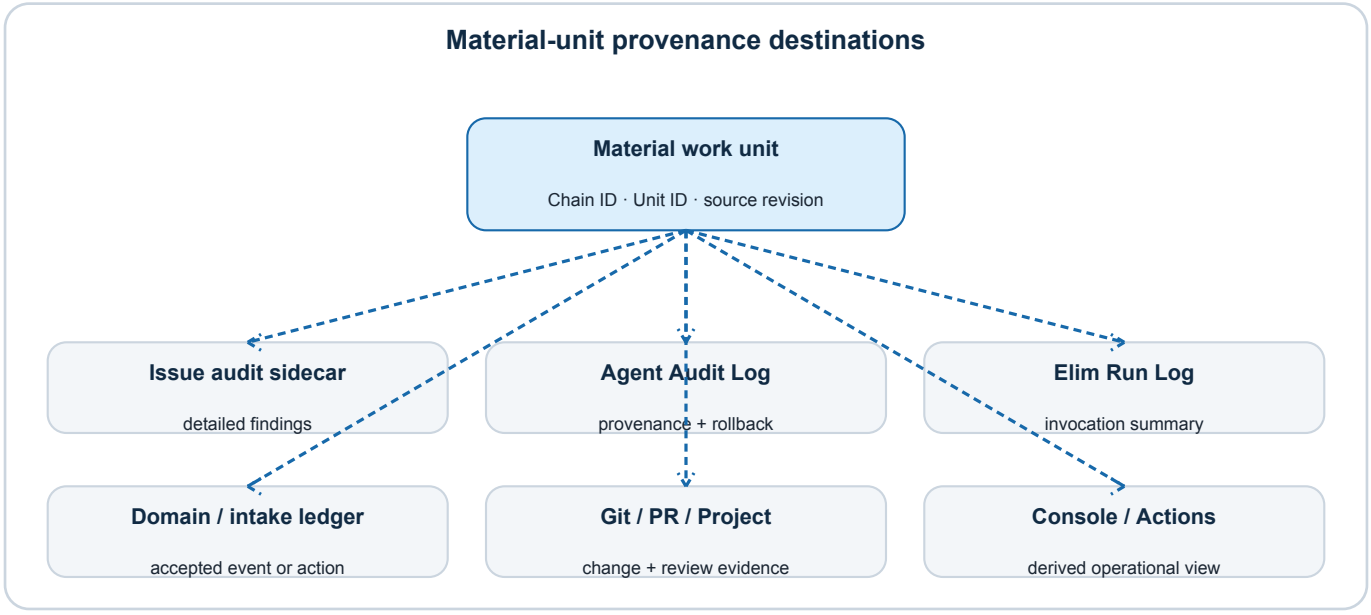

Figure: Material-unit provenance destinations. Arrows show data or control flow, not grants of authority.

Record	Purpose	Authority posture
Issue audit sidecar	Detailed T-audit and Change Audit findings for one issue.	Canonical audit history.
`AGENT_AUDIT_LOG.md`	Material persistent-agent/bot provenance and rollback.	Canonical operational provenance, not issue findings.
`ELIM_RUN_LOG.md`	One complete summary for every Elim invocation.	Canonical run-accounting record.
`SOURCE_MONITOR_LOG.md`	Accepted source-domain events and monitoring history.	Canonical domain-event record.
`CURRENT_AUDIT.md`	Abrupt-interruption continuation checkpoint.	Failsafe only; not liveness or history.
`.tmp/run-coordinator/elim-run-log-reconciliation.json`	Bounded obligations for launched Elim invocations that lack verified canonical run reports.	Host-local recovery state; never a substitute for the Run Log.
`.tmp/run-coordinator/reconciled-checkouts/`	Full dirty Elim checkouts retired only after canonical failed-run proof and Action Item resolution.	Preserved local evidence; never a reusable launch workspace.
Host-enriched Elim result and local chain projection	Actual trusted-host commit, synchronization evidence, cloud status, host status, and current-chain Elim runtime.	Host-local operational evidence; the original model result remains separately preserved.
`review-epochs.jsonl`	Append-only comprehensive-review boundaries and findings.	Canonical epoch evidence.
Current Integrity/Source reports	Replaceable latest finding set.	Current projection, not history.
`project-console-data`	Console-ready feeds, queue, packet, preserved inputs, bounded histories.	Generated projection.
GitHub Actions artifacts	Reproducibility and diagnostic evidence with defined retention.	Temporary evidence.
`.tmp/run-coordinator`	Local lease, control, attestations, downloads, JSONL, last result, and 128-event bootstrap-failure history.	Local operational state, not canonical project content.

The chain plan and completed manifest are retained as Actions artifacts for 30 days. Source Checker retains its complete machine-readable report for 30 days and bounded current history. Case and directive artifacts have bounded retention. The host reconciliation file retains at most 128 unique pending Chain IDs until exact proof permits removal; reaching the ceiling fails closed for human intervention. Git history provides additional change evidence but is not a substitute for an explicit event contract.

No-change and `not_due` results belong in bounded Actions or Console history. They generally do not append to the shared material log. A material detected or routed finding, project/external mutation, failure requiring intervention, or accepted source event receives durable provenance.

18. Console and local administration

The Console exposes:

- a project-wide automation-health alert on Overview;
- human-owned decisions and unresolved automation failures in Action Items;
- one card for each registered agent or bot;
- an error badge on the affected card;
- runbooks, current posture, schedule/due description, and log links;
- an Automation Administration view showing the chain, stage timing, retries, and rules;
- the latest parsed log entry separately above expandable earlier entries; and

- derived Integrity, Sources, Progress, Candidates, Publication, and Logs views.

The local controller binds only to `127.0.0.1:8766`. It accepts requests from the approved local Console origins, limits request bodies to 4,096 bytes, validates item IDs and reasons, and requires a random token for mutations. Supported requests include a normal run, comprehensive review, suppression, reprioritization, clearing a user-local override, and explicitly resolving a host Action Item with a human reason.

The controller and dispatcher use the same operating-system file lock and merge protocol when changing control state. Concurrent browser requests or a request arriving during a chain therefore cannot be lost to stale read-modify-write state. User-owned requests, suppressions, priorities, and resolution records remain distinct from dispatcher-owned runtime and failure fields.

This is same-user, trusted-workstation browser-request protection. It is not multi-user authentication and does not protect against a hostile process already running as the user. A control request is queued intent, not execution or authority. The coordinator records how it evaluated the request and the resulting Chain ID or rejection reason.

19. Security and privacy controls

The security model includes:

- least-privilege GitHub workflow permissions;
- pinned GitHub Action revisions;
- fixed allowlisted host executable paths and shell-free subprocess invocation;
- Keychain or GitHub Secrets credential ownership, with no plaintext fallback;
- repository-root containment, path normalization, `realpath` checks, symlink-escape rejection, regular-file checks, and exact path spelling validation;
- source revisions and SHA-256 hashes for manifests, inputs, queues, packets, and Review Epoch boundaries;
- strict JSON schemas and additional-property rejection;
- bounded standard-input transfer for immutable source-domain-event publication so workflow-controlled content cannot select an arbitrary local file;
- complete diff allowlists before any accepted watcher branch or retained log-rendering branch may supply executable code to a write-capable workflow;
- request timeouts, pagination and response ceilings, retries, backoff, pacing, and bounded provider calls;
- localhost binding, approved origins, mutation token, request-size limits, transactional control-state locking, and bounded control history;
- privacy-minimized intake events and ledgers;
- no contributor email, private body, rejected text, or unnecessarily reproduced vulgar, demeaning, or sensitive text in logs or artifacts;
- untrusted-content and prompt-injection treatment for public submissions and external pages; and
- human review for moderation, contact, source identity substitution, permanent disposition, publication, and configuration change.

Source monitoring is necessarily partial. A provider timeout, access restriction, blocked crawler, or bounded search result is not proof that no relevant development exists. Reports must preserve that uncertainty.

20. Validation and acceptance

The automation test surface includes:

- unit tests for due predicates, stage mapping, retry limits, queue construction, stable IDs, overrides, recovery, and selection;
- schema tests for manifests, queue, context, Elim results, intake records, and Review Epochs;
- context dependency, pin, freshness, byte-ceiling, and selected-record binding tests;
- usage baseline, reset-window, reserve, stale-snapshot, and final-gate tests;
- dirty workspace, branch divergence, isolated-checkout, lock contention, stale-lock, and interruption tests;
- public-intake privacy, cursor, idempotency, reply-validation, and injection-resistance tests;
- source-monitor request-bound, identity, access-restriction, and write-boundary tests;
- full runbook, registry, runtime-config, workflow, schema, output-path, schedule, failure-class, and Console drift checks;
- Project development-level and status-vocabulary validation;
- pull-request, data-branch, GitHub Project, Console, and publication readback; and
- PDF generation, text extraction, link, page, and rendered-layout verification for this reference product.

A successful Elim closeout proves at minimum:

1. the Chain ID and selected work-unit identity match;
2. the context packet and preserved inputs match their hashes and source revision;
3. the usage gate was passing at required boundaries and at final host validation;
4. the strict result schema and authority classification are valid;
5. reported validation contains no failed check;
6. material commit and synchronization evidence is real;
7. required issue, Agent Audit, Elim Run, intake, or Review Epoch records exist;
8. the handoff state is exact;
9. the isolated workspace is preserved or cleanly retired without changing the user's checkout; and
10. failures and human questions are visible in Action Items.

21. Change control, deployment, and rollback

An architecture change begins in the record that owns the rule. The same reviewed change must synchronize affected:

- Framework or agent-rule modules;
- persistent-role runbook;
- runtime JSON manifest;
- GitHub workflow;
- script and strict schema;
- queue/context registry;
- Console projection and controls;
- integrity drift checks;
- tests; and
- this non-authoritative specification when its description becomes stale.

Substantive automation-architecture changes trigger the required Change Audit and an off-cycle Review Epoch boundary. Configuration or rubric changes remain human-approved. A passing test suite does not authorize a reserved change.

Rollback uses a revert or another history-preserving reviewed commit on shared branches. A disposable bot branch may be safely replaced only under its runbook and `--force-with-lease`. Logs preserve the original action and later reversal instead of rewriting history.

22. Operational playbooks

22.1 Healthy no-op chain

All due bots complete or remain current; no integrity, intake, audit, development, candidate, or epoch unit is eligible. The coordinator publishes a complete no-op manifest and does not launch Elim. No shared material-log entry is required.

22.2 Deterministic watcher finds a change

The watcher creates a versioned proposed event and, where authorized, a narrow bot pull request. The chain preserves the event identity and hash. If the observation creates eligible LLM review, it enters the queue with the owning record and exact question. Merge by the allowlisted human project owner accepts the underlying bot change. The accepted event is rendered once into the source-domain and shared provenance records.

22.3 Source URL fails

The Source Checker retries a `GET` request and classifies the result. Access controls and transient provider failures remain distinct from breakage. A broken, mismatched, or review-required result enters Integrity and the eligible review queue. Any replacement must preserve source identity or receive substantive review and the applicable Change Audit.

22.4 Public submission arrives

The participation service emits a minimized flag. The collector reconciles it with the Discussion and cursor. Elim is launched only if the comment remains unassessed and other gates pass. It validates its assessment and any useful public reply, records the review cursor, and creates no permanent disposition.

22.5 Usage approaches the reserve

Elim reads the host attestation at the next required boundary. It finishes only the current atomic operation, validates and preserves it, begins no new work, records the continuation, and closes. A final nonpassing host reading prevents success.

22.6 Interactive checkout is dirty

The dispatcher does not stash, reset, or consume those changes. It uses its isolated automation workspace if the canonical remote boundary is valid. If the isolation boundary itself is unsafe or stale, the chain defers and raises an Action Item.

22.7 Elim terminates unexpectedly

The dispatcher preserves JSONL, last-message path, task identity, owner metadata, usage state, and a snapshot of the current handoff before a later checkout refresh could replace it. It records only artifacts that actually exist; a referenced artifact that later disappears fails closed. The dispatcher marks the invocation failed, records an Action Item and notification, and treats `CURRENT_AUDIT.md` only as recovery evidence. If the Codex process was spawned but no canonical Run Log report was verified, the dispatcher adds a host-local reconciliation record keyed to the original Chain ID and invocation. The file accepts at most 128 unique pending Chains.

The next current queue selects the pending set as a safety-class-0 `bot_failure` repair before ordinary work. Elim adds one complete failed-run report for each identified prior Chain, without converting incomplete analysis into a completed audit or project action, and gives the reconciliation invocation its own normal report. The dispatcher clears those records only after a completed result, an unchanged selected-state hash, and reviewed Git proof that each report was absent before the repair boundary and is newly present exactly once with every required field. The obligation does not expire with age. A partial,

duplicate, pre-existing, snapshot-changed, malformed, or noncompleted repair remains pending or fails closed. The same substantive unit does not resume until a fresh chain also validates its revision and durable continuation.

If the failed invocation also left the fixed isolated checkout dirty, the host must then invoke the distinct reconciled-checkout archive mode for the original Chain ID. The dispatcher independently rechecks the canonical report, empty pending queue, resolved Action Item, approved origin, checkout baseline, and dirty path set before moving the complete checkout into the private archive tree. A fresh chain can create a new fixed checkout only after that evidence-preserving move succeeds.

22.8 Human decision is required

Elim records the exact record, question, alternatives, and reason. It routes the matter to Action Items and may continue unrelated eligible work if the question does not make those inputs unreliable. It does not answer the reversed-control question or implement a permanent disposition.

23. Implementation status and known limitations

This section describes the reviewed implementation as of the date on the cover. It is not a waiver of any governing requirement.

Area	Current status	Residual limitation or required follow-up
Serialized chain	Implemented	Raw direct worker dispatch is diagnostic and does not reset the full-chain due boundary.
Deterministic-before-LLM order	Implemented	External monitoring remains bounded and non-exhaustive.
Host lease and diagnostic owner	Implemented	The owner record is evidence only; same-user local malware is outside the controller's threat boundary.
Isolated Elim workspace and trusted-host Git closeout	Implemented and integration-tested with a real temporary remote	Complete one production Elim unit before treating operational readiness as proven.
Candidate-research closeout	Implemented in schema and validator	Requires a real end-to-end candidate run before operational proof.
Queue/source watcher handoff	Implemented where structured source data is available	Provider artifacts remain bounded; accepted source-event rendering depends on the human merge event.
Context dynamic-expansion telemetry	Source and changed-file provenance implemented	There is no separate inventory of every canonical file read after packet construction.
Console overrides	Implemented and audited	Overrides cannot force eligibility or authority.
Usage reserve	Implemented at checkpoints and final gate; dormant zero-use windows are distinguished from anchored active windows	Cooperative checkpoint enforcement cannot guarantee an exact unused percentage.
Elim run/shared-log verification	Implemented for normal closeout and bounded post-spawn reconciliation	The repair path is proof-gated and requires a real interruption exercise before operational proof; task archiving remains interface housekeeping and is not a success criterion.
Host outcome projection	Local-first, same-chain reconciled	Static remote Console freshness may lag; cloud completion with an eligible Elim unit remains `host_pending` until matching host evidence arrives.
Bounded chain history	Implemented as configured or retained through explicit artifacts	Git history is not itself an immutable semantic event ledger.
Full runbook/runtime drift audit	Expanded	Newly added runtime fields must be added to the traceability check when configuration evolves.
Review Epochs	Implemented	The first post-architecture epoch remains necessary to establish operational stability.
Automation as a whole	Staged for controlled launch	A complete production chain and Elim unit remain the final acceptance demonstration.

The system should not be described as fully proven merely because configuration and tests pass. Operational proof requires one complete chain in which every due or not-due stage, queue item, context hash, usage attestation, isolated workspace, Elim result, pull request or data write, durable log event, and Console status reconciles under one Chain ID.

Appendix A. File and component crosswalk

Function	Owning or implementing files
Bootstrap and routing	`AGENTS.md`; `framework/FRAMEWORK.md`; `framework/AGENT_OPERATING_RULES.md`; `framework/CONTEXT_ROUTING.md`; `framework/context-routes.json`
Persistent-role authority	`framework/agents/README.md`; the seven registered runbooks
GitHub lifecycle	`framework/GITHUB_WORKFLOW.md`
Common automation rules	`framework/agent-rules/autonomous-execution.md`; `provenance-and-logging.md`; `validation-and-closeout.md`; `multi-agent.md`
Coordinator cloud runtime	`.github/workflows/run-coordinator-bot.yml`; `.github/run-coordinator-bot.json`; `scripts/run_coordinator.py`
Host dispatch/control	`scripts/run_chain_dispatcher.py`; `scripts/run_coordinator_control.py`; `scripts/check_codex_usage_reserve.py`; `.github/launchd/*.plist.example`
Queue and context	`scripts/arrp_context.py`; `scripts/build_elim_work_queue.py`; `scripts/select_elim_context_route.py`; `scripts/build_elim_context.py`
Elim result/execution	`framework/agents/elim-work-unit-result.schema.json`; `scripts/elim_execution.py`; `scripts/elim_execution_tools.py`
Deterministic workers	`.github/workflows/{case-monitor-bot,presidential-directives-bot,source-checker-bot,project-console-progress,project-integrity}.yml`; corresponding configs and scripts
Public intake	`framework/INTAKE_AGENT_PROCESS.md`; `scripts/collect_public_intake.py`; `scripts/record_intake_review.py`; `scripts/validate_elim_discussion_reply.py`; intake ledgers
Review Epochs	`scripts/record_review_epoch.py`; `research/review-epochs.jsonl`
Generated data	`scripts/publish_project_console_progress.py`; `project-console-data` branch
Source-event provenance	`.github/source-domain-event.schema.json`; `.github/workflows/source-domain-event-acceptance.yml`; `scripts/source_domain_events.py`; `scripts/publish_immutable_data_file.py`
Console	`framework/PROJECT_INTERFACE.md`; `scripts/build_horizon_review_console.py`; `research/horizon-review-console/`
Provenance	`framework/logs/CURRENT_AUDIT.md`; `AGENT_AUDIT_LOG.md`; `ELIM_RUN_LOG.md`; `SOURCE_MONITOR_LOG.md`; issue audit sidecars
Integrity	`scripts/audit_project_consistency.py`; `scripts/build_project_integrity_feed.py`; `framework/logs/PROJECT_INTEGRITY_REPORT.md`

Appendix B. Core invariants

1. A queue identifies work; it never creates authority.
2. A generated packet is not authority.
3. A deterministic bot never makes a substantive or permanent decision.
4. Only a human makes a permanent issue or candidate disposition.
5. Only a human answers the reversed-control question.
6. Rubrics may change only with human approval, never to produce a desired score.
7. Push-triggered chains never launch Elim.
8. Elim is the last substantive stage.

9 Integrity is the final deterministic input stage before Elim selection.

10 `CURRENT_AUDIT.md` is continuation state, not liveness.

11 The operating-system lease is host liveness.

12 User work is deferred around, never stashed, reset, absorbed, or overwritten.

13 The 15-percent reserve is protected; the 10-point target is soft.

14 T4 is not synonymous with Review Ready.

15 Score and `Runs` change only through the defined audit process.

16 Monitoring is independent of maturity and workflow status.

17 Deferred and Blocked require their specific hold predicates and explanations.

18 Private intake information and flagged content do not enter ordinary logs.

19 Proposed watcher events remain distinct from accepted events.

20 Generated data, current reports, and historical records must be labeled according to their actual authority and retention.

21 A launched Elim invocation never loses run accounting: normal closeout verifies its report, or a bounded proof-gated reconciliation obligation remains.

Appendix C. Glossary of stop conditions

Stop condition	Meaning
Authority stop	The action requires a human decision or lies outside the selected role.
Evidence stop	An indispensable source, identity, or factual basis is unavailable or contradictory.
Usage stop	A protected usage window is at or below reserve, the soft-target continuation rule is met, or official usage data is unavailable.
Context stop	Required canonical material is missing, stale, oversized, or inconsistent with the selected unit.
Repository stop	Revision, branch, worktree, merge, commit, push, or readback cannot be reconciled safely.
Validation stop	A required schema, test, integrity, audit, or publication check fails.
Authentication stop	Required GitHub, Project, provider, or host credential access is unavailable.
Human-review stop	The exact reserved question has been routed and must not be inferred.
Privacy stop	Safe handling, minimization, or public/private separation cannot be guaranteed.
Recovery stop	Prior incomplete work lacks a trustworthy exact continuation or current source boundary.

Appendix D. Project Integrity Bot check inventory

This table mirrors the runbook's plain-language check inventory. The runbook and checker implementation remain authoritative.

Check family	Deterministic boundary
Issue and proposal structure	Required sections, Issue Snapshot fields, and reader-visible Snapshot word-count warnings.
Area and topic routing	Canonical ownership and navigation placement.
Internal repository links	Missing or unsafe internal destinations.
Markdown heading anchors	Broken heading-fragment destinations.
Orphaned Markdown pages	Pages absent from required navigation or ownership.
Page metadata and heading hierarchy	Required front matter, allowed values, and structural heading order.
Cross-issue reference links	Identifier-linked ownership and target consistency.
GitHub record references	Valid issue, Project, pull-request, and canonical-record references.
GitHub Issue and Project synchronization	Registry, Issue wrapper, Project item, and controlled-field agreement.
Lifecycle-field coherence and explanations	Standard issue-page status, Project `Status`, `Development level`, monitoring details, and status-specific Deferred or Blocked reasons.
GitHub Pages synchronization	Successful deployment follows canonical `main` after the permitted grace interval.
Source and citation catalogs	Required fields, stable IDs, references, monitoring metadata, and catalog relationships.
Research placement	Area-owned and cross-project research uses the owning directory conventions.
Reader-facing language	Project-authored public language follows the neutral, reader-friendly conventions.
Tool-interface conventions	Console and other project-operated interfaces follow their owning standard.
Intake-workflow terminology	Public-submission records use the governed distinctions and do not imply unauthorized disposition.
Publication-disposition metadata	Every Markdown page is included in a print level or explicitly excluded with a reason, without conflict.
Print-assembly configuration	Manifest, ordering, level, and output conventions remain coherent.
Governing context registry	Registered paths, dependencies, hashes, profiles, byte ceilings, and complete governing coverage agree.
Persistent-agent runtime alignment	Registry, runbook, runtime manifest, workflow, schedule, environment, output paths, failure class, and chain order agree.
Source-domain event preservation and acceptance wiring	Watcher events, immutable proposal storage, attempt-specific artifacts, trusted-code guards, exact accepted deltas, and the human-merge acceptance boundary remain connected.
Structured-file and repository hygiene	JSON, JSONL, CSV, YAML, path, naming, generated-file, and repository conventions remain valid.